BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

SINH VIÊN THỰC HIỆN

HÀ NGUYỄN GIA KHANG
MSSV: 223746
LỚP: DH22KPM01

TÊN ĐỀ TÀI

THIẾT KẾ Ổ ĐIỆN THÔNG MINH ỨNG DỤNG CÔNG NGHỆ IOT

NGÀNH KỸ THUẬT PHẦN MỀM
Mã ngành: 7480103

Cần Thơ, tháng 3 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

HỌ TÊN TÁC GIẢ

HÀ NGUYỄN GIA KHANG
MSSV: 223746
LỚP: DH22KPM01

TÊN ĐỀ TÀI

THIẾT KẾ Ổ ĐIỆN THÔNG MINH ỨNG DỤNG CÔNG NGHỆ IOT

ĐỒ ÁN 2

NGÀNH KỸ THUẬT PHẦN MỀM

Mã ngành: 7480103

CÁN BỘ HƯỚNG DẪN

TRẦN VĂN THIỆN

Cần Thơ, tháng 3 năm 2026

LỜI CẢM TẠ

Chúng em xin chân thành cảm ơn quý Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Nam Cần Thơ đã tận tình giảng dạy, truyền đạt cho chúng em những kiến thức quý báu trong suốt quá trình học tập và thực hiện đề tài. Nhờ sự hướng dẫn và chỉ bảo tận tâm của quý Thầy Cô, chúng em đã có được nền tảng kiến thức chuyên ngành cần thiết để hoàn thành đồ án này.

Chúng em xin bày tỏ lòng biết ơn sâu sắc đến giảng viên hướng dẫn **Thầy Trần Văn Thiện**, người đã tận tình hướng dẫn, hỗ trợ và giải đáp những khó khăn mà nhóm gặp phải trong suốt quá trình thực hiện **Đồ án 2**. Sự quan tâm, góp ý và định hướng của thầy là nguồn động viên rất lớn, giúp nhóm chúng em hoàn thành bài báo cáo một cách tốt nhất.

Do kiến thức chuyên môn và kinh nghiệm thực tế của chúng em còn hạn chế nên trong quá trình thực hiện đồ án khó tránh khỏi những thiếu sót. Vì vậy, chúng em rất mong nhận được sự thông cảm cùng những ý kiến đóng góp quý báu từ quý Thầy Cô để nhóm có thể rút kinh nghiệm, bổ sung kiến thức và hoàn thiện bản thân hơn trong quá trình học tập sau này.

Cuối lời, chúng em xin kính chúc quý Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Nam Cần Thơ thật nhiều sức khỏe, hạnh phúc và thành công trong công tác giảng dạy.

Chúng em xin chân thành cảm ơn!

Cần Thơ, ngày tháng 3 năm 2026

Sinh viên thực hiện

HÀ NGUYỄN GIA KHANG

TRANG CAM KẾT

Chúng tôi xin cam kết đồ án tốt nghiệp này được hoàn thành dựa trên kết quả nghiên cứu của chúng tôi và các kết quả này chưa được sử dụng cho bất kỳ đồ án cùng cấp nào khác.

Cần Thơ, ngày tháng 3 năm 2026

Người thực hiện

HÀ NGUYỄN GIA KHANG

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

Cần Thơ, ngày .... tháng .....năm 2026

Giảng viên hướng dẫn

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU 1
1.1 Đặt vấn đề nghiên cứu 1
1.2 Mục tiêu nghiên cứu 1
1.2.1 Mục tiêu chung 1
1.2.2 Mục tiêu cụ thể 1
1.3 Phạm vi nghiên cứu 2
1.3.1 Không gian 2
1.3.2 Thời gian 2
1.3.3 Đối tượng nghiên cứu 2
1.3.4 Phạm vi của đề tài 2
1.4 Hiện trạng bài toán và yêu cầu đặt ra 3
1.5 Các bước thực hiện 3
1.6 Kết quả kỳ vọng 4
1.7 Bố cục báo cáo 4
CHƯƠNG 2: CƠ SỞ LÝ LUẬN VÀ PHƯƠNG PHÁP NGHIÊN CỨU 5
2.1 Tổng quan IoT và mô hình ổ điện thông minh 5
2.2 Nền tảng ESP32 6
2.3 Firebase Realtime Database 7
2.4 Flutter, Dart và tổ chức giao diện 8
2.5 WiFiManager, DHT11, relay, LCD I2C và buzzer 8
2.6 Phương pháp nghiên cứu và quy trình phát triển 9
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 10
3.1 Phân tích actor và yêu cầu hệ thống 10
3.2 Sơ đồ Use Case và đặc tả use case 12
3.3 Sơ đồ dòng dữ liệu, activity và sequence 16
3.4 Sơ đồ lớp, component, deployment và state 27
3.5 Thiết kế dữ liệu Firebase Realtime Database 32
CHƯƠNG 4: KẾT QUẢ XÂY DỰNG VÀ THẢO LUẬN 36
4.1 Môi trường phát triển và cấu trúc repository 36
4.2 Hiện thực ứng dụng Flutter 37
4.2.1 Giao diện nhập DEVICE_ID 38
4.2.2 Giao diện dashboard tổng quan 39
4.2.3 Giao diện điều khiển relay 40
4.2.4 Giao diện lịch sử hoạt động 41
4.2.5 Giao diện cài đặt thiết bị 42
4.3 Hiện thực firmware ESP32 43
4.4 Luồng dữ liệu đầu cuối và thảo luận kỹ thuật 45
CHƯƠNG 5: ĐÁNH GIÁ KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN 46
5.1 Kịch bản kiểm thử 46
5.2 Kết quả đạt được và giới hạn hiện tại 47
5.3 Hướng phát triển trong tương lai 48
KẾT LUẬN 49
TÀI LIỆU THAM KHẢO 50
PHỤ LỤC 51
Phụ lục A. Mẫu dữ liệu JSON minh họa 51
Phụ lục B. Ghi chú triển khai và lưu ý an toàn 53
Phụ lục C. Gợi ý cách sử dụng bộ sơ đồ đi kèm 53
Phụ lục D. Danh mục ảnh giao diện và phần cứng cần bổ sung 54

DANH SÁCH BẢNG

Bảng 1. Danh mục phần cứng chính của hệ thống. 7
Bảng 2. Danh mục nền tảng, ngôn ngữ và thư viện sử dụng. 8
Bảng 3. Danh sách actor và phạm vi tương tác. 10
Bảng 4. Bảng yêu cầu chức năng. 10
Bảng 5. Bảng yêu cầu phi chức năng. 11
Bảng 6. Đặc tả Use Case UC01 - Kiểm tra DEVICE_ID và truy cập thiết bị. 12
Bảng 7. Đặc tả Use Case UC02 - Xem dashboard tổng quan. 13
Bảng 8. Đặc tả Use Case UC03 - Điều khiển relay từ ứng dụng. 13
Bảng 9. Đặc tả Use Case UC04 - Xem lịch sử hoạt động. 14
Bảng 10. Đặc tả Use Case UC05 - Lưu cấu hình thiết bị. 14
Bảng 11. Đặc tả Use Case UC06 - Cấu hình Wi-Fi qua portal. 14
Bảng 12. Đặc tả Use Case UC07 - Điều khiển relay bằng nút nhấn cục bộ. 15
Bảng 13. Đặc tả Use Case UC08 - Cảnh báo nhiệt độ. 15
Bảng 14. Từ điển dữ liệu node info. 32
Bảng 15. Từ điển dữ liệu node settings. 32
Bảng 16. Từ điển dữ liệu node state (phần 1). 33
Bảng 17. Từ điển dữ liệu node state (phần 2). 33
Bảng 18. Từ điển dữ liệu node command/latest. 34
Bảng 19. Từ điển dữ liệu node history. 34
Bảng 20. Cấu trúc thư mục và mô-đun chính của repository. 36
Bảng 21. Bảng ánh xạ chân ESP32 và ngoại vi. 44
Bảng 22. Bảng timer và chu kỳ tác vụ trong firmware. 44
Bảng 23. Kịch bản kiểm thử và cách đánh giá. 46
Bảng 24. Phân loại hạn chế hiện tại và đề xuất cải tiến. 47
Bảng 25. Mẫu dữ liệu JSON minh họa cho devices/device01. 51
Bảng 26. Danh mục ảnh giao diện và phần cứng cần bổ sung. 54

DANH SÁCH HÌNH

Hình 1. Kiến trúc tổng thể hệ thống ổ điện thông minh. 5
Hình 2. Sơ đồ phần cứng và kết nối chân ESP32. 6
Hình 3. Sơ đồ Use Case tổng thể của hệ thống. 12
Hình 4. DFD mức ngữ cảnh cho hệ thống. 16
Hình 5. DFD mức 1 cho luồng xử lý chính. 16
Hình 6. Activity Diagram kiểm tra DEVICE_ID và mở trung tâm điều khiển. 18
Hình 7. Activity Diagram điều khiển relay từ ứng dụng. 19
Hình 8. Activity Diagram điều khiển relay bằng nút nhấn cục bộ. 21
Hình 9. Activity Diagram lưu cấu hình thiết bị. 22
Hình 10. Sequence Diagram khởi động ứng dụng và tải dashboard. 23
Hình 11. Sequence Diagram gửi lệnh điều khiển relay. 24
Hình 12. Sequence Diagram lưu cấu hình từ ứng dụng. 25
Hình 13. Sequence Diagram ghi nhận và hiển thị lịch sử sự kiện. 26
Hình 14. Class Diagram tầng mô hình và dữ liệu. 27
Hình 15. Class Diagram tầng giao diện và điều hướng. 28
Hình 16. Component Diagram của hệ thống. 29
Hình 17. Deployment Diagram khi triển khai thực tế. 29
Hình 18. State Diagram trạng thái kết nối của thiết bị ESP32. 30
Hình 19. State Diagram trạng thái cảnh báo nhiệt độ. 31
Hình 20. Cấu trúc cây dữ liệu Firebase Realtime Database. 32
Hình 21. Giao diện nhập DEVICE_ID. 39
Hình 22. Giao diện dashboard tổng quan. 40
Hình 23. Giao diện điều khiển relay. 41
Hình 24. Giao diện lịch sử hoạt động. 42
Hình 25. Giao diện cài đặt thiết bị. 43

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Diễn giải
IoT	Internet of Things - Internet vạn vật
ESP32	Vi điều khiển tích hợp Wi-Fi/Bluetooth của Espressif
RTDB	Realtime Database
SSE	Server-Sent Events
UI	User Interface
GPIO	General Purpose Input/Output
I2C	Inter-Integrated Circuit
DHT11	Cảm biến nhiệt độ và độ ẩm môi trường
AP	Access Point
JSON	JavaScript Object Notation
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete

CHƯƠNG 1: GIỚI THIỆU

1.1 Đặt vấn đề nghiên cứu

Trong các mô hình nhà ở thông minh quy mô nhỏ, nhu cầu thường gặp nhất không phải là những thuật toán quá phức tạp mà là khả năng giám sát được trạng thái thiết bị điện và can thiệp từ xa một cách đơn giản, chi phí thấp, dễ bảo trì. Ổ điện thông minh là dạng bài toán rất phù hợp để sinh viên triển khai đồ án vì nó kết hợp được cả ba lớp kiến thức quan trọng: phần cứng nhúng, dịch vụ dữ liệu thời gian thực và giao diện người dùng.

Dự án odien triển khai bài toán ổ điện thông minh theo mô hình ba lớp gồm ứng dụng Flutter, Firebase Realtime Database và firmware ESP32. Trong mô hình này, ứng dụng đóng vai trò giao diện điều khiển, Firebase giữ vai trò trung gian đồng bộ dữ liệu, còn ESP32 thực hiện điều khiển relay, cảm biến và hiển thị cục bộ. Cấu trúc này cho phép tách tương đối rõ lớp hiển thị, lớp dữ liệu và lớp chấp hành.

Luồng điều khiển của dự án được xây dựng theo nguyên tắc trạng thái hiển thị phải phản ánh trạng thái đã được phần cứng xác nhận. Theo đó, ứng dụng không cập nhật ngay trạng thái relay trên giao diện sau khi gửi lệnh mà chờ ESP32 poll node command/latest, áp lệnh lên relay thật rồi mới ghi state mới lên Firebase. Cơ chế này làm rõ sự phân biệt giữa lệnh yêu cầu và trạng thái thực thi thực tế.

Hệ thống đồng thời duy trì cơ chế điều khiển cục bộ bằng nút nhấn vật lý ngay cả trong trường hợp mất kết nối mạng. Yêu cầu này cho thấy đề tài không được xây dựng theo hướng phụ thuộc hoàn toàn vào cloud, mà vẫn bảo toàn lớp điều khiển tại chỗ để phục vụ nhu cầu sử dụng cơ bản của thiết bị.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu chung

Mục tiêu chung của đề tài là xây dựng hệ thống ổ điện thông minh ứng dụng công nghệ IoT, cho phép thực hiện đồng thời hai nhóm chức năng chính gồm điều khiển - giám sát thiết bị điện từ xa và vận hành tại chỗ trên phần cứng nhúng. Hệ thống được thiết kế trên nền tảng ESP32, ứng dụng Flutter và Firebase Realtime Database nhằm hình thành một mô hình tích hợp giữa thiết bị phần cứng, cơ sở dữ liệu thời gian thực và ứng dụng điều khiển. Trong mô hình này, thiết bị ESP32 đảm nhiệm chức năng thu thập dữ liệu cảm biến, xử lý trạng thái relay,

tiếp nhận lệnh điều khiển và duy trì khả năng vận hành cục bộ; trong khi đó, ứng dụng Flutter đóng vai trò giao diện tương tác, hỗ trợ người dùng theo dõi tình trạng thiết bị, gửi lệnh điều khiển và cập nhật cấu hình từ xa. Trên cơ sở đó, đề tài hướng đến việc hình thành một hệ thống có tính đồng bộ dữ liệu, có khả năng mở rộng, đồng thời bảo đảm tính liên tục trong vận hành ngay cả khi kết nối mạng không ổn định.

1.2.2 Mục tiêu cụ thể

Thứ nhất, xây dựng firmware cho vi điều khiển ESP32 nhằm tổ chức hoạt động điều khiển bốn relay, thu thập dữ liệu từ cảm biến nhiệt độ - độ ẩm DHT11, cảm biến ánh sáng, điều khiển còi buzzer cảnh báo và hiển thị thông tin lên màn hình LCD I2C. Khối chức năng này giữ vai trò trung tâm trong việc thực hiện xử lý tín hiệu đầu vào, điều khiển tải đầu ra và phản hồi trạng thái hoạt động của hệ thống.

Thứ hai, thiết kế cấu trúc dữ liệu trên Firebase Realtime Database theo mô hình cây JSON có phân tầng rõ ràng, bao gồm các nhóm dữ liệu info, settings, state, command và history. Cách tổ chức này nhằm bảo đảm tính nhất quán giữa dữ liệu mô tả thiết bị, dữ liệu cấu hình, dữ liệu trạng thái tức thời, lệnh điều khiển và lịch sử vận hành, từ đó tạo điều kiện thuận lợi cho việc đồng bộ giữa ứng dụng và phần cứng.

Thứ ba, phát triển ứng dụng Flutter theo hướng đa màn hình, bao gồm các trang Tổng quan, Điều khiển, Lịch sử và Cài đặt, đồng thời bổ sung cơ chế nhập DEVICE_ID trước khi truy cập hệ thống. Cấu trúc giao diện này cho phép người dùng tiếp cận hệ thống theo đúng định danh thiết bị, quan sát trạng thái hiện hành, thực hiện tác vụ điều khiển và thay đổi tham số vận hành.

Thứ tư, xây dựng lớp DeviceRepository làm tầng trung gian truy xuất dữ liệu, sử dụng kết hợp giao thức HTTP REST và cơ chế Server-Sent Events. Giải pháp này được lựa chọn nhằm phù hợp với đặc điểm luồng dữ liệu của hệ thống, trong đó các thao tác đọc - ghi cấu hình và lệnh điều khiển được thực hiện theo yêu cầu, còn dữ liệu trạng thái được cập nhật theo thời gian thực.

Thứ năm, bảo đảm cơ chế điều khiển cục bộ vẫn duy trì được khi thiết bị mất kết nối mạng. Yêu cầu này có ý nghĩa quan trọng trong vận hành thực tế, vì

hệ thống không chỉ hoạt động như một mô hình điều khiển từ xa mà còn phải bảo đảm tính sẵn sàng tại chỗ. Khi kết nối được khôi phục, các thay đổi trạng thái cần được đồng bộ ngược lên Firebase để duy trì tính thống nhất của dữ liệu.

Thứ sáu, xây dựng báo cáo phân tích và thiết kế hệ thống trên cơ sở bám sát hiện trạng mã nguồn và mô hình triển khai thực tế. Nội dung báo cáo bao gồm các sơ đồ use case, activity, sequence, class, component, deployment và state diagram nhằm mô tả đầy đủ các góc nhìn chức năng, dữ liệu, tương tác và triển khai của hệ thống.

1.3 Phạm vi nghiên cứu

1.3.1 Không gian

Không gian áp dụng của đề tài là các bài toán ổ điện hoặc cụm tải dân dụng quy mô nhỏ trong phòng khách, bàn học, phòng làm việc hoặc khu vực sinh hoạt gia đình. Hệ thống được thiết kế cho bốn kênh relay, phù hợp với các tải như đèn bàn, quạt, bộ sạc hoặc thiết bị phụ trợ công suất vừa và nhỏ.

1.3.2 Thời gian

Báo cáo này được biên soạn trên cơ sở snapshot repository công khai và firmware hiện có tại thời điểm tháng 03 năm 2026. Điều đó có nghĩa là các nhận xét về cấu trúc chương trình, sơ đồ lớp, sơ đồ trình tự và đánh giá chức năng đều phản ánh hiện trạng phiên bản đang được lưu trên nhánh chính của dự án, không giả định các tính năng chưa xuất hiện trong mã nguồn.

1.3.3 Đối tượng nghiên cứu

Đối tượng nghiên cứu gồm ba thành phần: (1) phần cứng ESP32 và ngoại vi, (2) mô hình dữ liệu thời gian thực trên Firebase, và (3) ứng dụng Flutter làm giao diện điều khiển - giám sát. Ngoài ra, báo cáo còn xem xét tương tác giữa người dùng với hệ thống, giữa nút nhấn vật lý với phần cứng, và giữa dịch vụ đám mây với thiết bị nhúng.

1.3.4 Phạm vi của đề tài

Trong phạm vi hiện tại, hệ thống tập trung vào điều khiển bật/tắt relay, xem dashboard, đồng bộ cài đặt, ghi lịch sử và cảnh báo nhiệt độ môi trường theo ngưỡng. Dự án chưa triển khai xác thực người dùng hoàn chỉnh, chưa có cơ chế phân quyền nhiều tài khoản, chưa dùng token hoặc rules bảo mật nâng cao cho

Firebase, và cũng chưa hiện thực luật tự động phức tạp cho chế độ auto. Những phần này được xem là hướng phát triển, không phải chức năng hoàn tất ở phiên bản phân tích hiện tại.

1.4 Hiện trạng bài toán và yêu cầu đặt ra

Ở các ổ cắm điện thông thường, người dùng chỉ có hai lựa chọn: đến trực tiếp vị trí thiết bị để bật/tắt, hoặc dùng thêm các bộ hẹn giờ, công tắc thủ công, thiết bị chuyển mạch độc lập. Cách tiếp cận này thiếu khả năng quan sát trạng thái từ xa, khó lưu lịch sử sự kiện và không tạo được nền tảng để tích hợp các luật thông minh về sau.

Nhiều sản phẩm ổ điện thông minh thương mại trên thị trường có giá thành cao, phụ thuộc ứng dụng đóng, khó tùy biến và ít giá trị học thuật đối với sinh viên khi cần tìm hiểu cơ chế hoạt động bên trong. Đề tài vì vậy hướng tới giải pháp tự xây dựng dựa trên ESP32 và Firebase, vừa tiết kiệm chi phí, vừa giúp người học làm chủ toàn bộ luồng dữ liệu từ phần cứng lên giao diện.

Từ góc nhìn yêu cầu hệ thống, bài toán đặt ra năm yêu cầu cốt lõi: thứ nhất, ứng dụng phải xác định đúng thiết bị thông qua DEVICE_ID; thứ hai, trạng thái dashboard phải phản ánh trạng thái thực tế của relay; thứ ba, lệnh điều khiển từ xa phải có cơ chế ghi nhận và phản hồi; thứ tư, hệ thống phải lưu được lịch sử sự kiện; và thứ năm, khi mất mạng, nút nhấn vật lý vẫn phải điều khiển được relay. Đây cũng là cơ sở để xây dựng các use case và sơ đồ hoạt động ở các chương sau.

1.5 Các bước thực hiện

Quy trình triển khai đề tài được xây dựng theo hướng tăng dần độ hoàn thiện. Trước tiên là khảo sát bài toán và chốt phạm vi chức năng để tránh dàn trải. Sau đó, nhóm tập trung xây dựng phần cứng nền tảng, ánh xạ GPIO và kiểm tra khả năng điều khiển relay, buzzer, LCD, DHT11 và nút nhấn cục bộ trên ESP32. Song song, mô hình dữ liệu Firebase được định nghĩa để bảo đảm các node điều khiển, trạng thái và lịch sử tách bạch rõ ràng.

Sau giai đoạn phần cứng - dữ liệu, ứng dụng Flutter được xây dựng từ entry point đến các màn hình cụ thể, trong đó ConnectionGatePage là lớp chặn đầu vào để chọn DEVICE_ID, còn HomeShell là shell điều hướng cho bốn tab chức năng. Cuối

cùng, phần repository được kết nối với Firebase bằng HTTP REST và SSE, từ đó hoàn thiện luồng end-to-end giữa giao diện, cloud và phần cứng.

- Khảo sát bài toán và chốt phạm vi chức năng.
- Thiết kế mô hình dữ liệu Firebase Realtime Database.
- Lập trình firmware ESP32 và kiểm tra ngoại vi.
- Xây dựng ứng dụng Flutter với bốn mô-đun chức năng chính.
- Tích hợp luồng command, state, settings và history.
- Xây dựng sơ đồ phân tích - thiết kế và biên soạn báo cáo.

1.6 Kết quả kỳ vọng

Kết quả kỳ vọng của đề tài là hình thành một mô hình ổ điện thông minh có khả năng bật/tắt bốn relay, giám sát môi trường, đồng bộ dữ liệu thời gian thực và cung cấp cơ sở kiến trúc đủ rõ cho việc mở rộng về sau. Bên cạnh sản phẩm chạy thử, báo cáo còn hướng tới việc làm rõ vì sao hệ thống chọn luồng command/latest -> ESP32 -> state thay vì cập nhật trực tiếp trạng thái trên giao diện.

Ngoài sản phẩm phần mềm - phần cứng, báo cáo còn đặt mục tiêu chuẩn hóa bộ sơ đồ use case, activity, sequence, class, component, deployment và state để làm tài liệu mô tả kỹ thuật. Các sơ đồ này được dùng như cơ sở để bảo trì, mở rộng và đối chiếu với mã nguồn ở các giai đoạn tiếp theo.

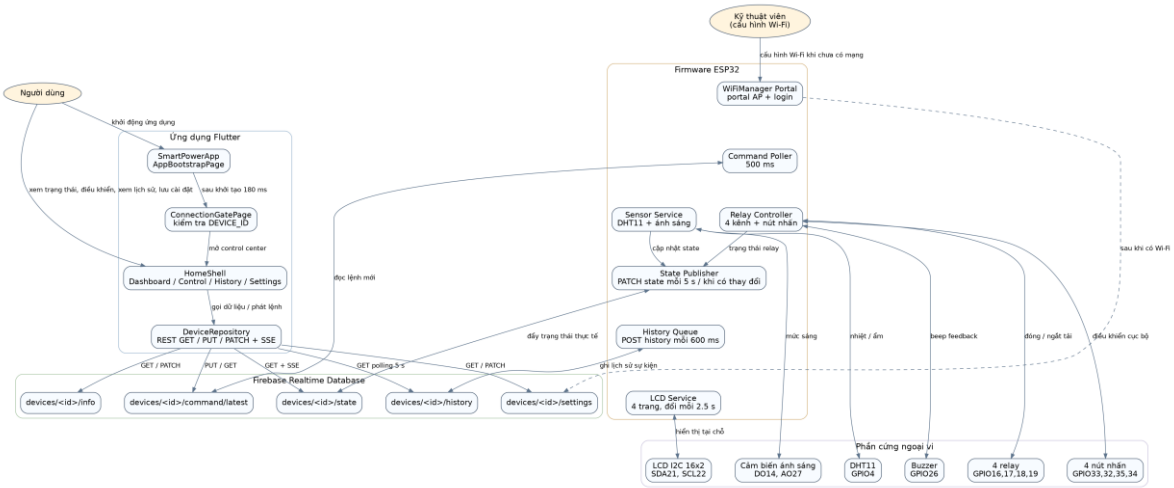
CHƯƠNG 2: CƠ SỞ LÝ LUẬN VÀ PHƯƠNG PHÁP NGHIÊN CỨU

2.1 Tổng quan IoT và mô hình ổ điện thông minh

Internet of Things cho phép các thiết bị vật lý thu thập dữ liệu, truyền dữ liệu và nhận lệnh điều khiển qua mạng, từ đó hình thành các hệ thống giám sát - điều khiển phân tán ở quy mô nhỏ đến lớn (Google, 2025b).

Khác với mạch relay bật/tắt đơn thuần, hệ thống trong đề tài có thêm lớp dữ liệu thời gian thực, lớp giao diện và lớp lịch sử sự kiện. Vì vậy, khi phân tích đề tài cần xem đây là một hệ thống thông tin IoT thu nhỏ, trong đó trạng thái thiết bị, lệnh điều khiển và dữ liệu cảm biến cùng được quản lý trên một mô hình dữ liệu thống nhất.

Trong kiến trúc hiện tại, hệ thống được chia thành ba lớp chính. Lớp thứ nhất là client Flutter chịu trách nhiệm hiển thị và nhận thao tác người dùng. Lớp thứ hai là Firebase Realtime Database dùng để lưu trữ dữ liệu dạng cây JSON và đồng bộ trạng thái giữa các thành phần. Lớp thứ ba là firmware ESP32 trực tiếp điều khiển phần cứng và phát sinh trạng thái vận hành.



Hình 1. Kiến trúc tổng thể hệ thống ổ điện thông minh.

Hình 1 mô tả kiến trúc tổng thể của hệ thống. Trong kiến trúc này, DeviceRepository đóng vai trò lớp trung gian giữa giao diện Flutter và Firebase, qua đó gom các thao tác GET, PUT, PATCH và lắng nghe SSE về một đầu mối xử lý thống nhất.

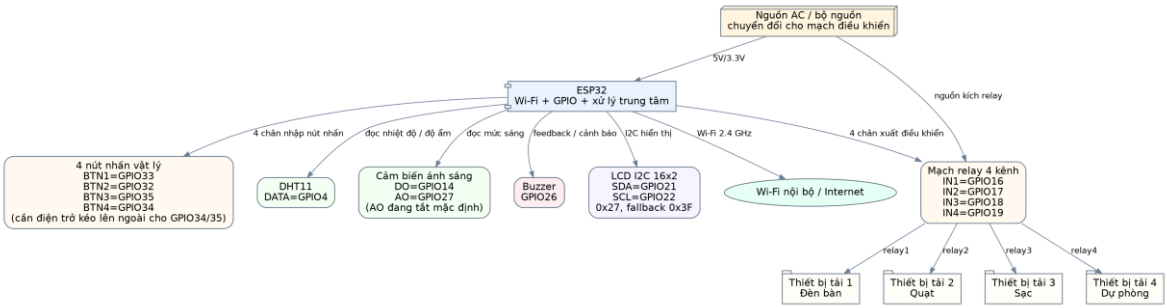
Ở phía firmware, các khối chức năng có thể được nhận diện thành các mô-đun như WiFiManager portal, command poller, sensor service, relay controller, history queue và LCD service. Cách phân rã này hỗ trợ việc xây dựng component diagram và sequence diagram dựa trên đúng trách nhiệm của từng khối xử lý.

2.2 Nền tảng ESP32

ESP32 là vi điều khiển tích hợp Wi-Fi, Bluetooth và nhiều chân GPIO, thường được sử dụng trong các bài toán IoT nhờ chi phí phù hợp và hệ sinh thái thư viện phong phú (Espressif Systems, 2025). Trong đề tài này, ESP32 đồng thời đảm nhiệm điều khiển relay, đọc cảm biến, quét nút nhấn, giao tiếp LCD và thực hiện các yêu cầu HTTP tới Firebase.

Trong firmware, phần ánh xạ GPIO được khai báo tập trung ở đầu tệp chương trình, qua đó mô tả rõ relay, nút nhấn, DHT11, buzzer, cảm biến ánh sáng và LCD đang được nối với các chân nào trên ESP32. Cách tổ chức này giúp việc kiểm tra phần cứng và hiệu chỉnh sơ đồ nối chân thuận tiện hơn.

Trong phần chú thích mã nguồn, tác giả cũng ghi rõ GPIO34 và GPIO35 không có pull-up nội trên ESP32, vì vậy cần điện trở kéo lên ngoài nếu dùng nút nhấn active-low. Đây là chi tiết thực tế rất đáng giá trong báo cáo, vì nó cho thấy dự án không chỉ dừng ở mức mô tả phần mềm mà còn chú ý đến tính khả thi khi lắp ráp phần cứng. (Hà Nguyễn Gia Khang, 2026).



Hình 2. Sơ đồ phần cứng và kết nối chân ESP32.

Hình 2 mô tả sơ đồ phần cứng ở mức khối. Khác với sơ đồ nguyên lý điện tử chi tiết, hình này tập trung vào mối quan hệ chức năng giữa ESP32 và các ngoại vi: relay, nút nhấn, cảm biến, buzzer, LCD và mạng Wi-Fi. Cách biểu diễn này phù hợp cho một báo cáo phân tích - thiết kế hệ thống, vì người đọc có thể nhìn nhanh được vai trò của từng khối và đường truyền dữ liệu/chân GPIO chính.

Một chi tiết kỹ thuật cần nhấn mạnh là analog light read trên GPIO27 hiện đang bị tắt mặc định trong firmware do ADC2 có thể xung đột với Wi-Fi trên ESP32. Điều đó cho thấy dự án đã nhận diện được ràng buộc phần cứng thực tế và chọn giải pháp an toàn hơn là dùng ngõ digital DO14 cho logic ngưỡng trong phiên bản hiện tại.

Bảng 1. Danh mục phần cứng chính của hệ thống.

Thành phần	Nhóm	Vai trò trong hệ thống
ESP32	Vi điều khiển trung tâm	Kết nối Wi-Fi, điều khiển GPIO, xử lý HTTP với Firebase
Relay 4 kênh	Chấp hành	Đóng/ngắt bốn tải điện độc lập
Nút nhấn vật lý	Điều khiển cục bộ	Cho phép thao tác tại chỗ khi mất mạng
DHT11	Cảm biến môi trường	Đọc nhiệt độ và độ ẩm xung quanh
Cảm biến ánh sáng	Cảm biến phụ	Cung cấp trạng thái sáng/tối và dự phòng cho tự động hóa
Buzzer	Phản hồi âm thanh	Beep khi thao tác và khi vượt ngưỡng nhiệt độ
LCD I2C 16x2	Hiển thị tại chỗ	Hiển thị trạng thái Wi-Fi, relay và cảm biến
Nguồn và Wi-Fi nội bộ	Hạ tầng	Cấp nguồn và truyền dữ liệu giữa ESP32 với Firebase

2.3 Firebase Realtime Database

Firebase Realtime Database là cơ sở dữ liệu NoSQL do Google cung cấp, lưu trữ dữ liệu theo cấu trúc cây JSON và hỗ trợ đồng bộ trạng thái gần thời gian thực giữa các client đang kết nối (Google, 2025b; Google, 2025c). Đặc tính này phù hợp với bài toán ổ điện thông minh vì trạng thái relay và cảm biến cần được phản ánh nhanh trên dashboard.

Trong phiên bản hiện tại, ứng dụng Flutter không dùng trực tiếp SDK Firebase mà sử dụng gói http kết hợp REST và SSE để giao tiếp với Realtime Database. Cách triển khai này làm rõ luồng truy cập dữ liệu ở tầng repository và phù hợp với cấu trúc dashboard đang được dùng trong dự án.

Tuy nhiên, việc dùng RTDB qua REST cũng đặt ra yêu cầu quan trọng về thiết kế dữ liệu. Các node phải đủ phẳng để client đọc nhanh, tên trường phải ổn định, và phân biệt rõ node điều khiển với node trạng thái thực tế. Đó là lý do vì sao dự án tách command/latest và state thành hai khu vực khác nhau: command là nơi nhận yêu cầu, còn state là nơi ghi nhận kết quả thực thi thực tế từ ESP32.

2.4 Flutter, Dart và tổ chức giao diện

Flutter là bộ SDK giao diện đa nền tảng do Google phát triển, cho phép xây dựng ứng dụng từ một codebase thống nhất với hệ thống widget phong phú và khả năng triển khai trên nhiều nền tảng (Google, 2025a). Trong repository odien, Flutter được dùng để tổ chức các trang kết nối thiết bị, dashboard, điều khiển, lịch sử và cài đặt.

Giao diện Flutter trong dự án được tổ chức theo hướng ưu tiên điều hướng rõ ràng và chia sẻ dữ liệu tập trung. HomeShell nhận DeviceDashboard từ repository rồi phân phối cho bốn màn hình chức năng; đồng thời giao diện chuyển đổi giữa NavigationBar và NavigationRail tùy theo độ rộng màn hình.

Ngoài các trang chức năng, dự án còn có lớp giao diện dùng chung qua app_chrome.dart và bảng màu qua AppTheme/AppPalette. Nhờ đó, các panel, badge trạng thái, thẻ chỉ số và tiêu đề khu vực được thống nhất phong cách. Từ góc độ báo cáo, điều này cho thấy hệ thống đã vượt qua mức "UI tạm dùng" và có ý thức tổ chức thành design system nhỏ để tái sử dụng.

2.5 WiFiManager, DHT11, relay, LCD I2C và buzzer

WiFiManager là thư viện hỗ trợ ESP32 và ESP8266 mở cổng cấu hình Wi-Fi khi thiết bị chưa kết nối được vào mạng đã lưu, từ đó cho phép người dùng hoặc kỹ thuật viên chọn SSID và nhập mật khẩu ngay trên captive portal (tzapu, 2025). Trong đề tài, thư viện này được sử dụng như lớp cấu hình mạng ban đầu cho thiết bị.

DHT11 là cảm biến nhiệt độ - độ ẩm môi trường phổ biến, có đầu ra số và thường được dùng trong các mô hình học thuật hoặc giám sát môi trường cơ bản (ASAIR, 2025). Trong hệ thống này, DHT11 chỉ phản ánh điều kiện môi trường xung quanh chứ không đại diện cho nhiệt độ tiếp điểm điện hay relay.

Trong kiến trúc phần cứng, relay giữ vai trò chấp hành đóng/ngắt tải, buzzer tạo phản hồi âm thanh, còn LCD I2C cung cấp lớp hiển thị tại chỗ cho trạng thái Wi-Fi, relay và cảm biến. Tổ hợp này giúp thiết bị vẫn có khả năng vận hành và quan sát cơ bản ngay cả khi người dùng chưa mở ứng dụng.

Bảng 2. Danh mục nền tảng, ngôn ngữ và thư viện sử dụng.

Nền tảng / thư viện	Nhóm	Vai trò

Flutter / Dart	Phát triển ứng dụng	Xây dựng giao diện, điều hướng và tương tác người dùng
http	Client networking	Gửi GET/PUT/PATCH/POST tới Firebase và mở SSE stream
intl	Định dạng dữ liệu	Định dạng ngày giờ và thông điệp hiển thị
google_fonts	Giao diện	Định nghĩa typography cho theme
ESP32 Arduino Core	Nền tảng nhúng	Biên dịch và nạp firmware cho ESP32
WiFiManager	Cấu hình mạng	Portal captive để cấu hình Wi-Fi khi mất kết nối
ArduinoJson	Xử lý dữ liệu	Đóng gói/giải mã JSON cho Firebase
DHT	Thư viện cảm biến	Đọc nhiệt độ và độ ẩm từ DHT11
LiquidCrystal_I2C	Hiển thị	Điều khiển LCD 16x2 qua I2C

2.6 Phương pháp nghiên cứu và quy trình phát triển

Báo cáo này sử dụng phương pháp nghiên cứu theo hướng kết hợp gồm phân tích mã nguồn hiện trạng, đối chiếu với tài liệu kỹ thuật chính thức và chuẩn hóa lại dưới dạng sơ đồ, bảng đặc tả và mô tả hệ thống (Hà Nguyễn Gia Khang, 2026; Google, 2025a; Google, 2025b; Espressif Systems, 2025). Cách tiếp cận này giúp nội dung báo cáo bám sát hệ thống thật thay vì mô tả theo giả định.

Ở mức triển khai, dự án được phát triển theo hướng lặp tăng dần: firmware được hoàn thiện từ các chức năng relay và cảm biến, sau đó bổ sung publish state, history queue và captive portal; phía ứng dụng cũng được mở rộng dần từ luồng khởi tạo ban đầu sang bốn màn hình chức năng và tầng repository riêng.

Một nguyên tắc xuyên suốt trong quá trình nghiên cứu là phân biệt rõ giữa “dữ liệu yêu cầu” và “dữ liệu thực tế”. Lệnh điều khiển từ app chỉ là yêu cầu gửi lên command/latest; còn trạng thái hiển thị cuối cùng phải đến từ state do ESP32 xác nhận. Chính nguyên tắc này chi phối hầu hết các quyết định phân tích ở các chương sau.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Phân tích actor và yêu cầu hệ thống

Để thiết kế đúng sơ đồ use case, trước hết cần xác định rõ actor của hệ thống. Ở đây actor không nhất thiết chỉ là người dùng cuối, mà còn có thể là tác nhân kỹ thuật tác động từ bên ngoài vào hệ thống như kỹ thuật viên cấu hình Wi-Fi hoặc nút nhấn cục bộ đóng vai trò nguồn kích hoạt sự kiện. Việc tách actor giúp tránh mô tả sai phạm vi trách nhiệm giữa ứng dụng, cloud và phần cứng.

Sau khi xác định actor, báo cáo tiếp tục phân loại yêu cầu hệ thống theo hai nhóm: yêu cầu chức năng và yêu cầu phi chức năng. Yêu cầu chức năng trả lời câu hỏi hệ thống phải làm được gì; còn yêu cầu phi chức năng trả lời hệ thống phải làm việc tốt đến mức nào, ổn định ra sao, có trung thực dữ liệu không và có giữ được local control khi mất mạng hay không.

Bảng 3. Danh sách actor và phạm vi tương tác.

Actor	Phạm vi tương tác
Người dùng	Nhập DEVICE_ID, mở dashboard, điều khiển relay, xem lịch sử, lưu cài đặt
Kỹ thuật viên	Cấu hình Wi-Fi qua captive portal khi thiết bị chưa có mạng
Nút nhấn cục bộ	Kích hoạt thay đổi relay trực tiếp tại phần cứng
ESP32	Thực thi lệnh, đẩy state, ghi history, đồng bộ settings
Firebase RTDB	Lưu trữ và đồng bộ command, state, settings, history

Bảng 4. Bảng yêu cầu chức năng.

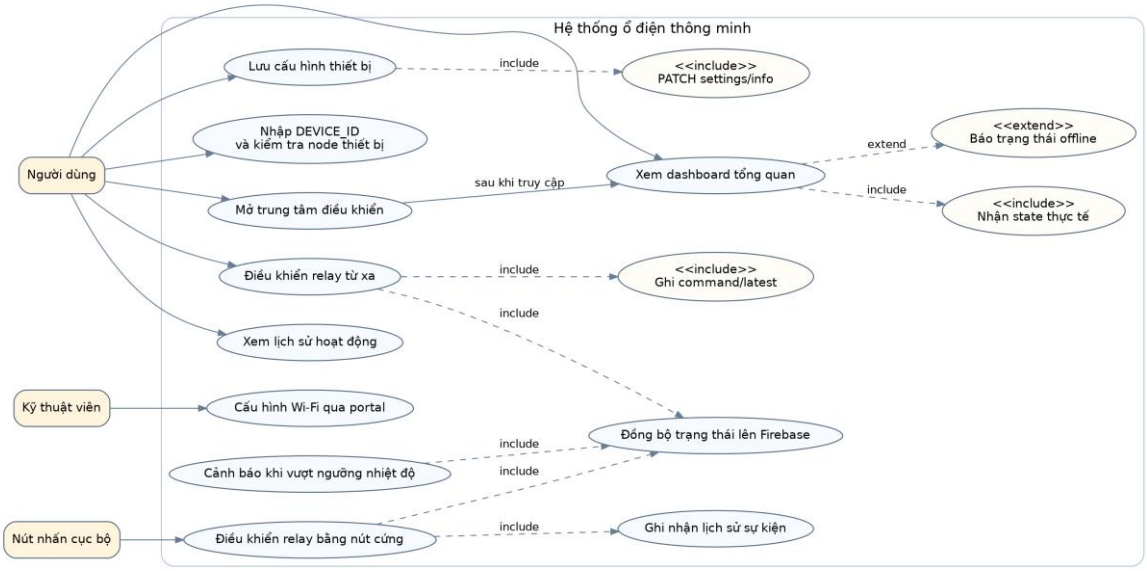
Mã	Yêu cầu chức năng
FR01	Nhập DEVICE_ID và kiểm tra sự tồn tại của node thiết bị trước khi cho phép truy cập
FR02	Hiển thị dashboard gồm trạng thái online/offline, môi trường, relay và lệnh mới nhất
FR03	Cho phép gửi lệnh bật/tắt relay từ ứng dụng thông qua command/latest
FR04	Đồng bộ trạng thái relay thực tế từ node state bằng SSE
FR05	Cho phép xem lịch sử sự kiện và lọc theo source hoặc relay

FR06	Cho phép cập nhật tên thiết bị, vị trí, mode, buzzer, ngưỡng nhiệt và tên relay
FR07	Cho phép ESP32 đọc settings định kỳ và áp dụng cấu hình mới
FR08	Cho phép nút nhấn vật lý điều khiển relay độc lập với mạng
FR09	Ghi lịch sử boot và relay_change lên Firebase
FR10	Cảnh báo khi nhiệt độ môi trường vượt ngưỡng cấu hình

Bảng 5. Bảng yêu cầu phi chức năng.

Mã	Yêu cầu phi chức năng
NFR01	Giao diện phải phản ánh trạng thái thực tế của ESP32, không chỉ trạng thái dự đoán từ app
NFR02	Hệ thống phải duy trì local control qua nút nhấn ngay cả khi mất kết nối Internet
NFR03	Repository phải tự nối lại SSE sau khi đứt kết nối mạng tạm thời
NFR04	Dữ liệu phải được tổ chức rõ ràng theo cây JSON để dễ mở rộng
NFR05	Giao diện phải có responsive layout giữa NavigationBar và NavigationRail
NFR06	Firmware phải giới hạn chu kỳ đọc cảm biến, poll command và publish state để tránh nghẽn
NFR07	Báo cáo phải nêu rõ giới hạn an toàn: DHT11 không đại diện nhiệt độ tiếp điểm relay
NFR08	Kiến trúc phải đủ mở để sau này bổ sung bảo mật, tự động hóa và thông báo đẩy

3.2 Sơ đồ Use Case và đặc tả use case



Hình 3. Sơ đồ Use Case tổng thể của hệ thống.

Use Case tổng thể tập trung vào những chức năng mà hệ thống hiện thật sự có trong repo: xác thực thiết bị theo DEVICE_ID, mở trung tâm điều khiển, xem dashboard, điều khiển relay, xem lịch sử, lưu cài đặt, cấu hình Wi-Fi và điều khiển cục bộ bằng nút cứng. Các use case "include" được dùng để nhấn mạnh một số hành vi luôn đi kèm, chẳng hạn điều khiển relay luôn bao hàm việc ghi lệnh vào command/latest, còn xem dashboard luôn gắn với việc nhận state thực tế từ Firebase.

Điểm quan trọng trong use case của đề tài là sự hiện diện của nút nhấn cục bộ như một actor riêng. Điều này nhắc người đọc rằng sản phẩm không vận hành hoàn toàn theo mô hình cloud-first. Trong hệ thống này, phần cứng vẫn có quyền tạo ra sự kiện mà không cần đợi ứng dụng, sau đó mới đồng bộ ngược ra cloud và dashboard.

Bảng 6. Đặc tả Use Case UC01 - Kiểm tra DEVICE_ID và truy cập thiết bị.

Trường	Nội dung
Mã use case	UC01
Tên use case	Kiểm tra DEVICE_ID và truy cập thiết bị
Actor chính	Người dùng
Tiền điều kiện	Ứng dụng đã khởi động và người dùng đang ở ConnectionGatePage

Luồng chính	Người dùng nhập DEVICE_ID; app tạo DeviceRepository; repository đọc info, state, settings; nếu có node dữ liệu thì tạo ConnectionSummary và cho phép mở HomeShell
Luồng thay thế	DEVICE_ID rỗng -> báo lỗi; không có node -> báo chưa tìm thấy; có node nhưng heartbeat cũ -> vẫn báo reachable nhưng trạng thái offline
Hậu điều kiện	Repository được chọn làm thiết bị hoạt động của phiên làm việc

Bảng 7. Đặc tả Use Case UC02 - Xem dashboard tổng quan.

Trường	Nội dung
Mã use case	UC02
Tên use case	Xem dashboard tổng quan
Actor chính	Người dùng
Tiền điều kiện	Đã vào HomeShell
Luồng chính	watchDashboard() bootstrap info/settings/command/state; repository mở SSE tới state; DashboardPage hiển thị online, mode, cảnh báo, chỉ số môi trường và bốn relay
Luồng thay thế	SSE gián đoạn -> repository tự thử nối lại sau 2 giây; nếu lastSeen cũ thì hiển thị offline
Hậu điều kiện	Người dùng quan sát được trạng thái thực tế của hệ thống

Bảng 8. Đặc tả Use Case UC03 - Điều khiển relay từ ứng dụng.

Trường	Nội dung
Mã use case	UC03
Tên use case	Điều khiển relay từ ứng dụng
Actor chính	Người dùng
Tiền điều kiện	Thiết bị reachable, đang ở ControlPage
Luồng chính	Người dùng đổi trạng thái một relay; ControlPage gọi sendRelayCommand(); Firebase ghi command/latest; ESP32 poll thấy command mới, áp lệnh, ghi history, publish state; dashboard nhận state mới
Luồng thay thế	command invalid -> ESP32 ghi ack ignored_target hoặc ignored_action; UI giữ nguyên relay theo state cũ
Hậu điều kiện	Trạng thái relay thực tế và lịch sử được đồng bộ

Bảng 9. Đặc tả Use Case UC04 - Xem lịch sử hoạt động.

Trường	Nội dung
Mã use case	UC04
Tên use case	Xem lịch sử hoạt động
Actor chính	Người dùng
Tiền điều kiện	Thiết bị đã có dữ liệu tại node history
Luồng chính	HistoryPage lắng nghe watchHistory(); repository GET history mỗi 5 giây; danh sách HistoryEntry được parse và hiển thị theo thời gian giảm dần
Luồng thay thế	Chưa có history -> hiển thị trạng thái rỗng; người dùng có thể lọc theo nguồn hoặc relay
Hậu điều kiện	Người dùng theo dõi được các thay đổi relay, boot và nguồn kích hoạt

Bảng 10. Đặc tả Use Case UC05 - Lưu cấu hình thiết bị.

Trường	Nội dung
Mã use case	UC05
Tên use case	Lưu cấu hình thiết bị
Actor chính	Người dùng
Tiền điều kiện	Đang ở SettingsPage
Luồng chính	Người dùng sửa tên, vị trí, mode, buzzer, ngưỡng và tên relay; _dirty được bật; nhấn Lưu; repository PATCH info và settings; ESP32 đọc settings định kỳ và áp dụng
Luồng thay thế	Lưu lỗi -> hiển thị SnackBar lỗi; người dùng có thể Undo để khôi phục dữ liệu từ dashboard
Hậu điều kiện	Cấu hình mới được lưu trên cloud và sẵn sàng cho ESP32 đọc lại

Bảng 11. Đặc tả Use Case UC06 - Cấu hình Wi-Fi qua portal.

Trường	Nội dung
Mã use case	UC06
Tên use case	Cấu hình Wi-Fi qua portal

Actor chính	Kỹ thuật viên
Tiền điều kiện	Thiết bị chưa kết nối được Wi-Fi đã lưu hoặc cần cấu hình lại
Luồng chính	ESP32 mở AP + portal; kỹ thuật viên đăng nhập; chọn SSID, nhập mật khẩu; lưu cấu hình; thiết bị kết nối lại vào Wi-Fi
Luồng thay thế	Sai thông tin hoặc timeout -> portal tiếp tục chờ cho đến khi retry hoặc hết thời gian cấu hình
Hậu điều kiện	Thiết bị chuyển sang trạng thái có mạng và bắt đầu đồng bộ cloud

Bảng 12. Đặc tả Use Case UC07 - Điều khiển relay bằng nút nhấn cục bộ.

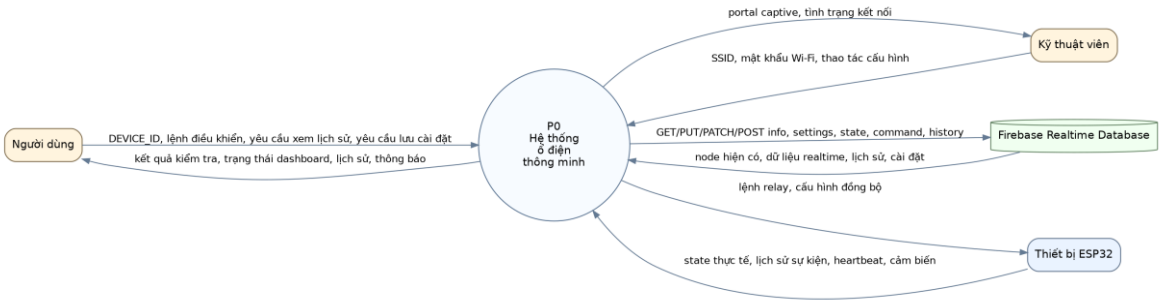
Trường	Nội dung
Mã use case	UC07
Tên use case	Điều khiển relay bằng nút nhấn cục bộ
Actor chính	Nút nhấn cục bộ
Tiền điều kiện	Thiết bị đang cấp nguồn
Luồng chính	scanButtons() debounce tín hiệu; nếu là BTN1 thì yêu cầu nhấn giữ 220 ms; sau đó handleLocalButton() gọi applyRelayState(), enqueueHistoryEvent() và publishState(true)
Luồng thay thế	Nhiễu hoặc nhấn quá ngắn -> bỏ qua
Hậu điều kiện	Relay đổi trạng thái tại chỗ và app sẽ nhận state mới khi có mạng

Bảng 13. Đặc tả Use Case UC08 - Cảnh báo nhiệt độ.

Trường	Nội dung
Mã use case	UC08
Tên use case	Cảnh báo nhiệt độ
Actor chính	ESP32
Tiền điều kiện	Có dữ liệu nhiệt độ từ DHT11 và đã cấu hình tempLimit
Luồng chính	readSensors() cập nhật temperatureC; updateAlarmState() so sánh với tempLimitC; nếu vừa vượt ngưỡng thì beep 180 ms; state published với alarm=true và overTempLock=true

Luồng thay thế	Nếu buzzerEnabled=false thì app vẫn thấy alarm nhưng xem như buzzerMuted
Hậu điều kiện	Dashboard hiển thị trạng thái cảnh báo dựa trên nhiệt độ môi trường

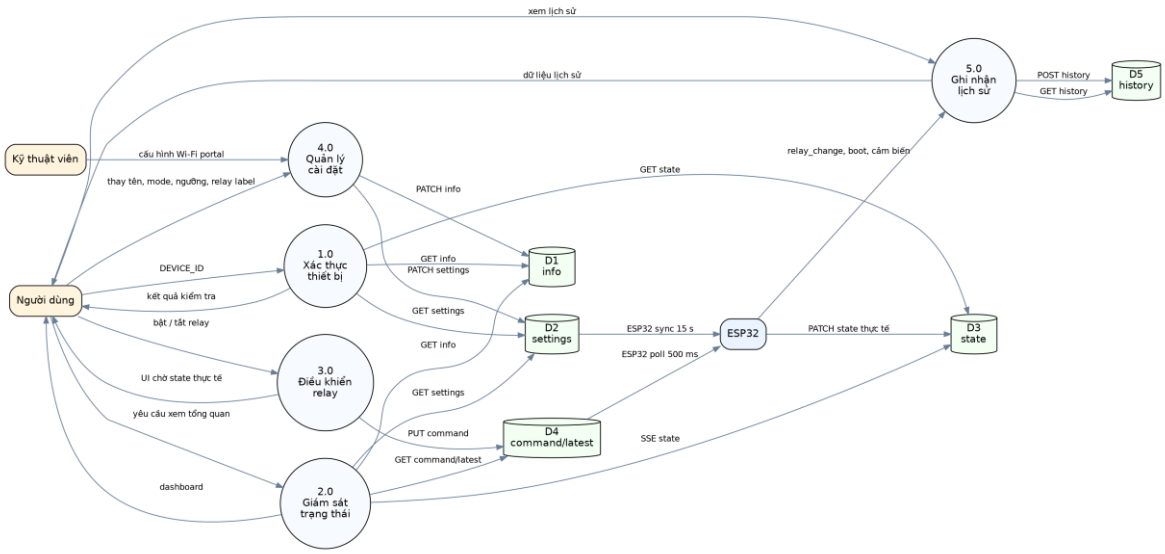
3.3 Sơ đồ dòng dữ liệu, activity và sequence



Hình 4. DFD mức ngữ cảnh cho hệ thống.

DFD mức ngữ cảnh đặt toàn bộ hệ thống vào một tiến trình duy nhất để làm rõ biên giao tiếp với các tác nhân bên ngoài. Ở mức này, người dùng gửi DEVICE_ID, lệnh điều khiển và yêu cầu truy vấn; kỹ thuật viên gửi thông tin cấu hình Wi-Fi; còn Firebase và ESP32 trao đổi dữ liệu trạng thái, lịch sử và cấu hình với hệ thống.

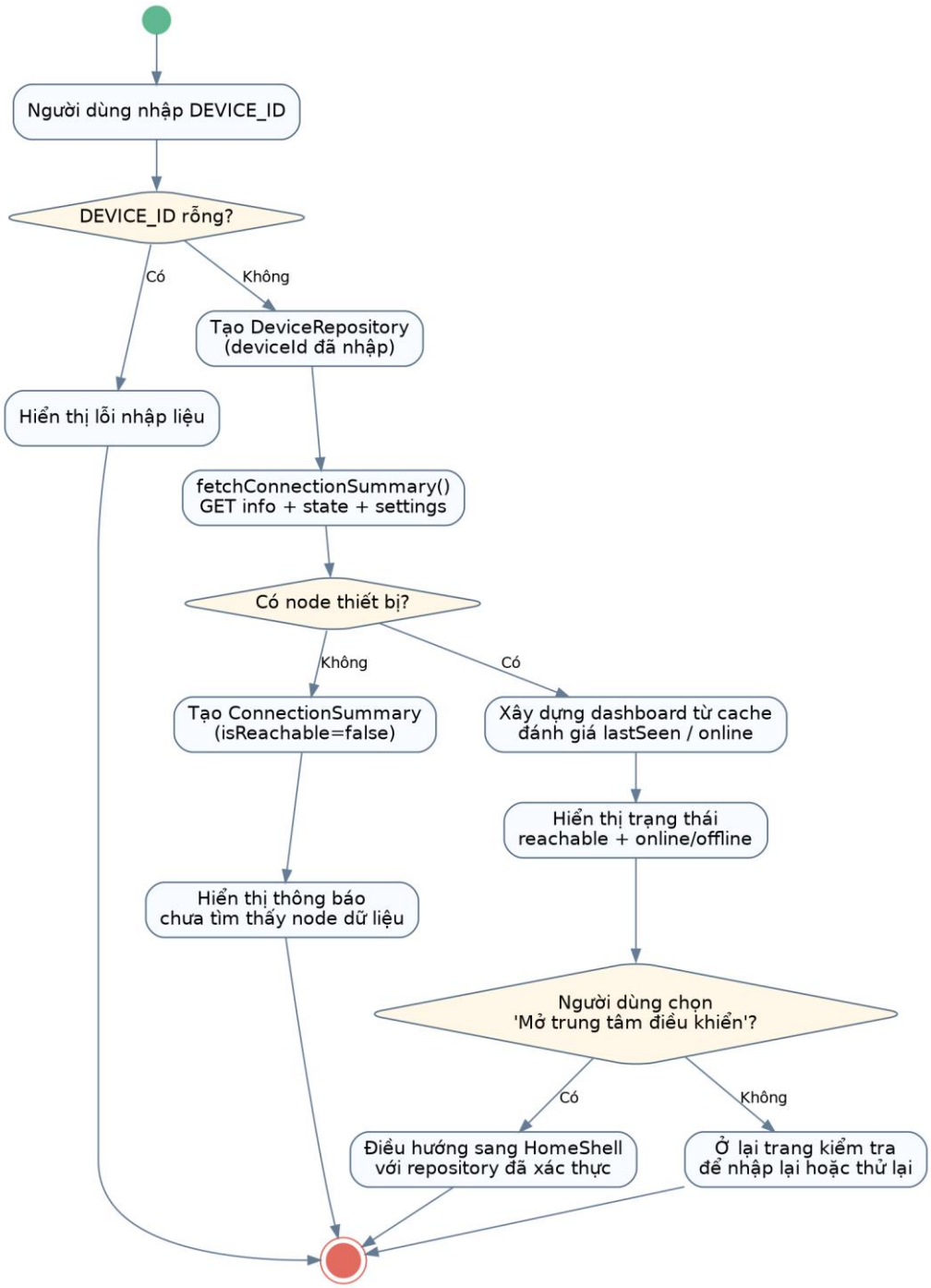
Sơ đồ này giúp xác định ranh giới quan trọng: Firebase không phải là nơi ra quyết định cuối cùng về trạng thái relay mà chỉ là nơi trung gian lưu trữ và đồng bộ. Quyết định áp lệnh và tạo state thực tế vẫn thuộc về ESP32.



Hình 5. DFD mức 1 cho luồng xử lý chính.

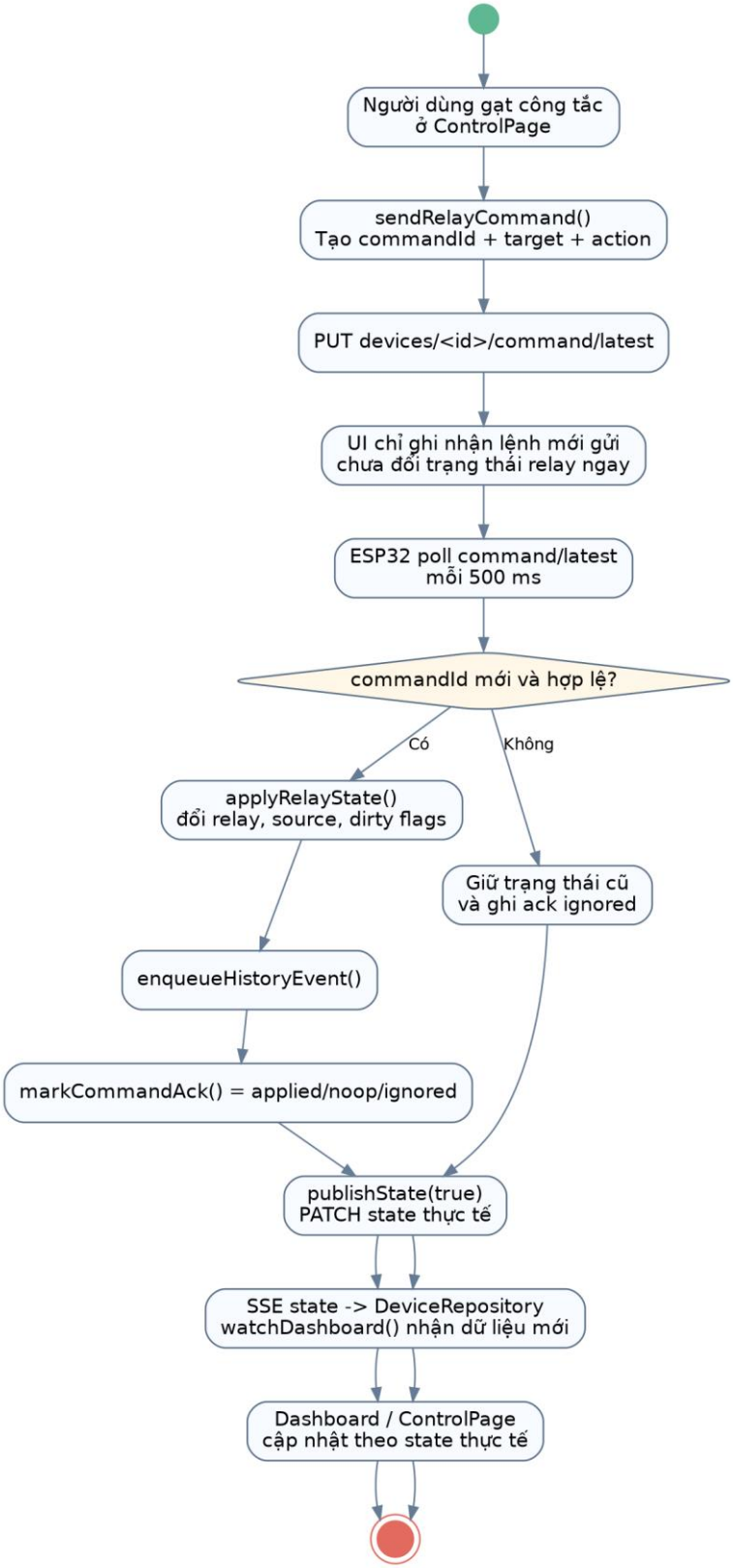
DFD mức 1 tách hệ thống thành năm xử lý chính: xác thực thiết bị, giám sát trạng thái, điều khiển relay, quản lý cài đặt và ghi nhận lịch sử. Năm xử lý này tương ứng khá chặt với các mô-đun đã tồn tại trong repo và firmware, nhờ vậy biểu đồ không chỉ mang tính lý thuyết mà còn bám sát mã nguồn.

Trong đó, xử lý điều khiển relay cho thấy rõ luồng app -> command/latest -> ESP32 -> state, còn xử lý giám sát trạng thái thể hiện cơ chế đọc state bằng SSE và refresh các node tĩnh theo chu kỳ. Đây là hai luồng lõi nhất của hệ thống.



Hình 6. Activity Diagram kiểm tra DEVICE_ID và mở trung tâm điều khiển.

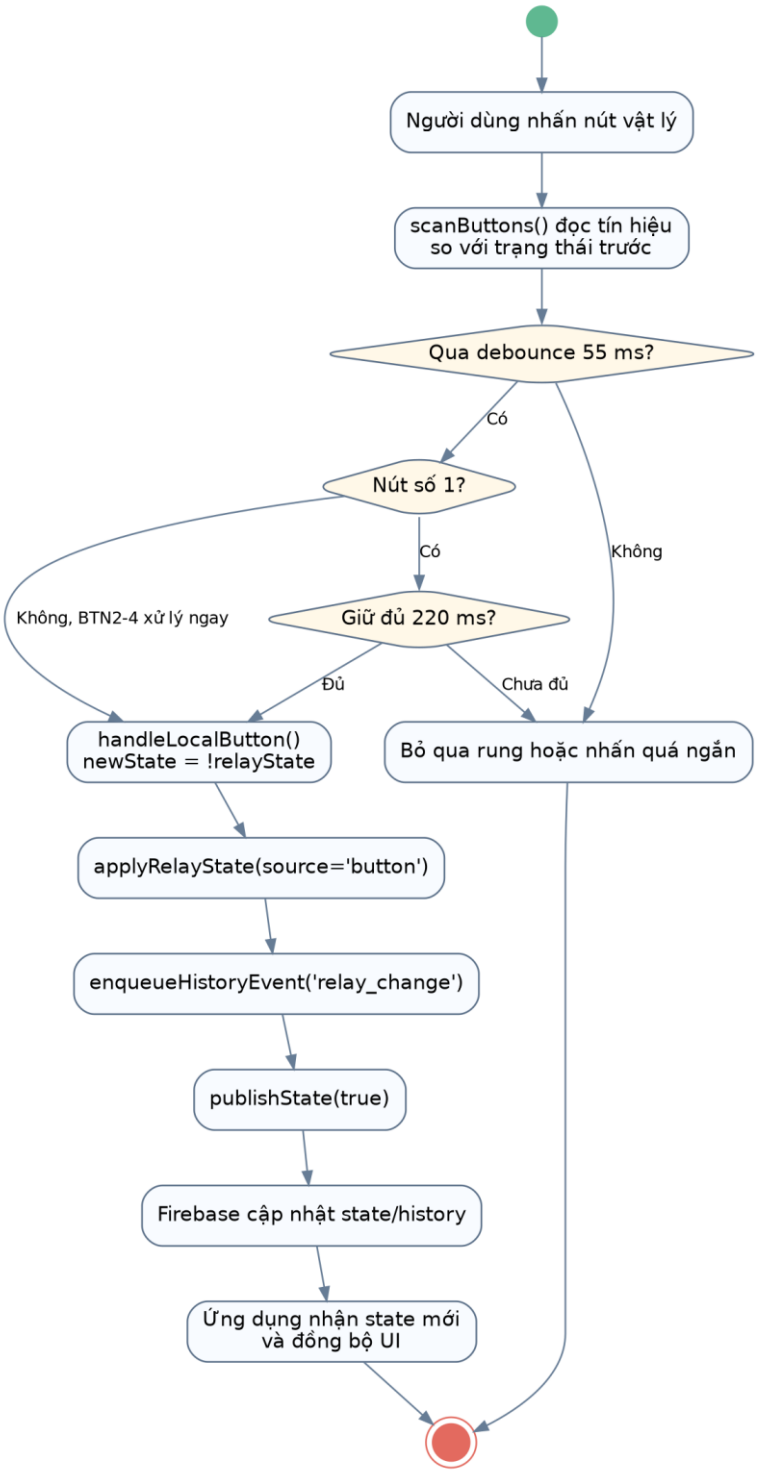
Activity Diagram ở Hình 6 mô tả tương tác đầu tiên giữa người dùng và ứng dụng. Điều quan trọng ở sơ đồ này là app không kiểm tra một node duy nhất mà đọc đồng thời info, state và settings để đánh giá xem thiết bị có tồn tại hay không. Nếu thiết bị reachable nhưng lastSeen cũ, app vẫn cho phép mở HomeShell, từ đó người dùng có thể quan sát trạng thái offline thay vì bị chặn hoàn toàn.



Hình 7. Activity Diagram điều khiển relay từ ứng dụng.

Activity Diagram điều khiển relay nhấn mạnh một nguyên tắc thiết kế then chốt: UI chỉ phát lệnh, còn thay đổi cuối cùng phải đến từ state do ESP32 publish. Vì vậy,

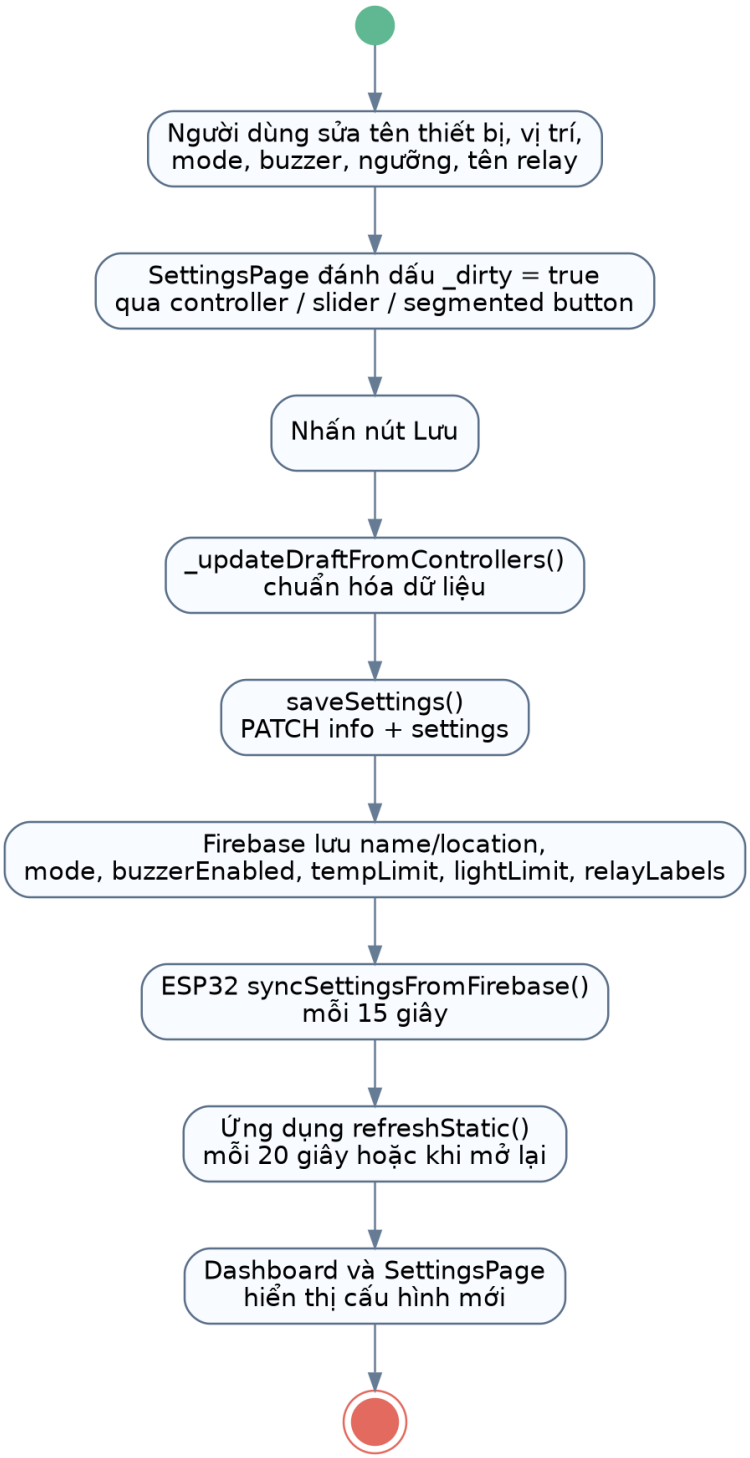
sau khi app ghi command/latest, ControlPage chưa đổi ngay công tắc theo kiểu optimistic update, mà chờ tới khi firmware poll lệnh, áp relay thật và PATCH state mới.



Hình 8. Activity Diagram điều khiển relay bằng nút nhấn cục bộ.

Sơ đồ hoạt động cho nút nhấn cục bộ làm rõ vì sao hệ thống vẫn vận hành được khi mất mạng. Nút nhấn không cần đi qua app hay cloud; nó tác động trực tiếp vào relay thông qua handleLocalButton(). Khi có mạng, state và history mới được đẩy lên

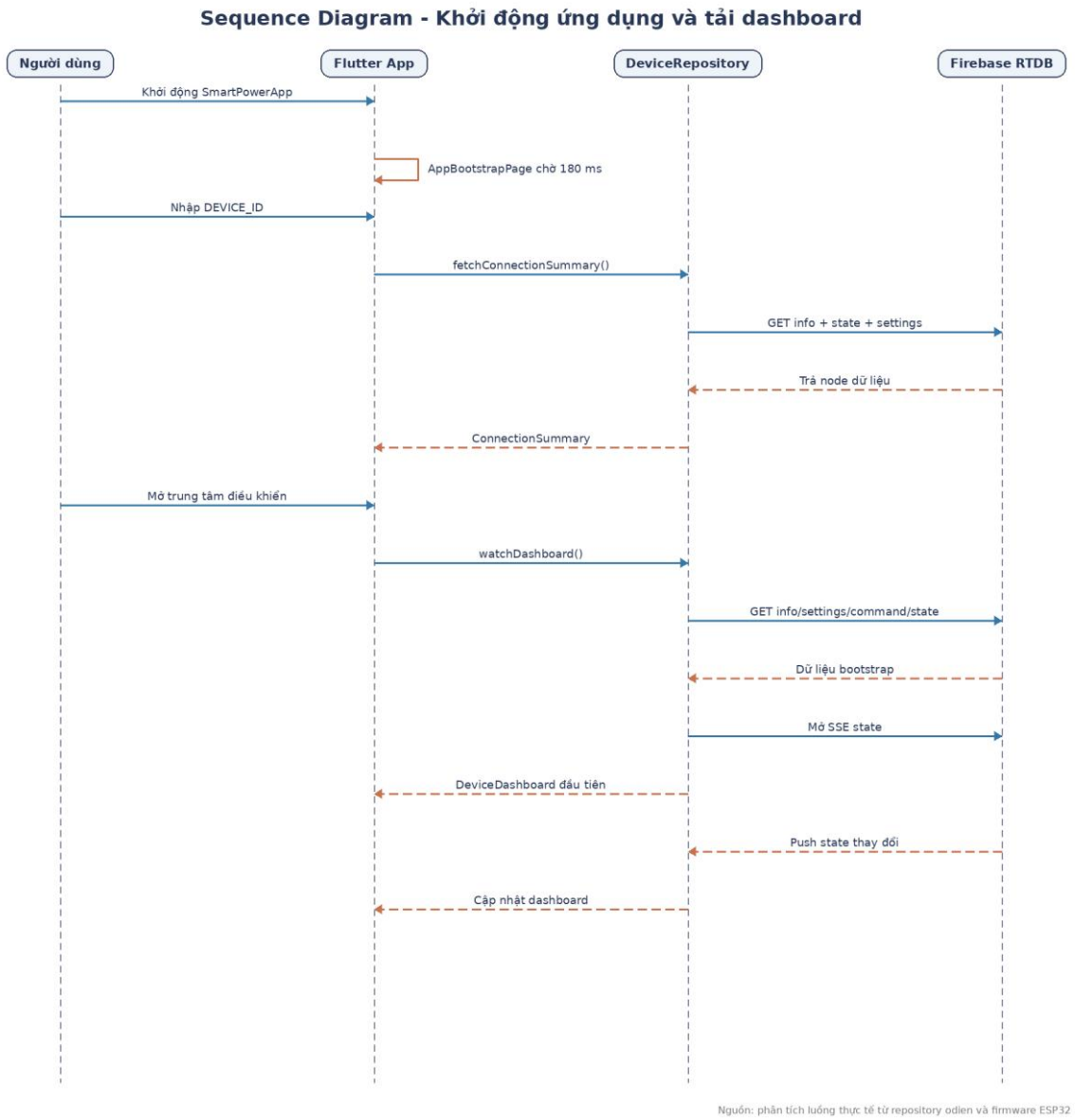
Firebase để app đồng bộ theo sau. Điều này giúp local control luôn có ưu tiên cao hơn đối với thao tác cơ bản.



Hình 9. Activity Diagram lưu cấu hình thiết bị.

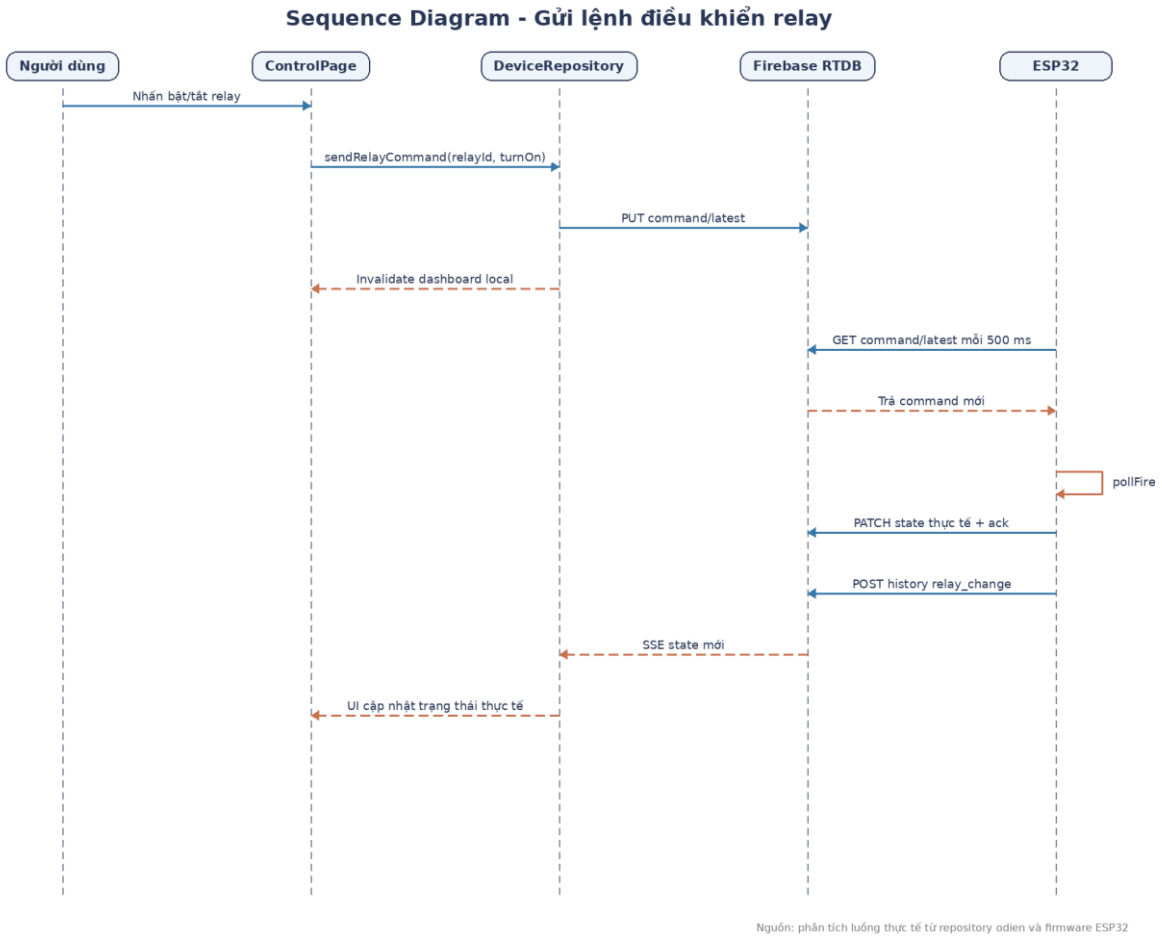
Luồng lưu cài đặt cho thấy tính chất eventual consistency của hệ thống. Ứng dụng lưu dữ liệu lên info và settings ngay lập tức, nhưng ESP32 chỉ đọc settings theo

chu kỳ 15 giây. Do đó, người thiết kế cần hiểu rằng cài đặt có thể đã được lưu trên cloud nhưng thiết bị chưa áp dụng trong cùng thời điểm. Đây là điểm cần ghi rõ trong báo cáo để tránh kỳ vọng sai về thời gian đáp ứng.



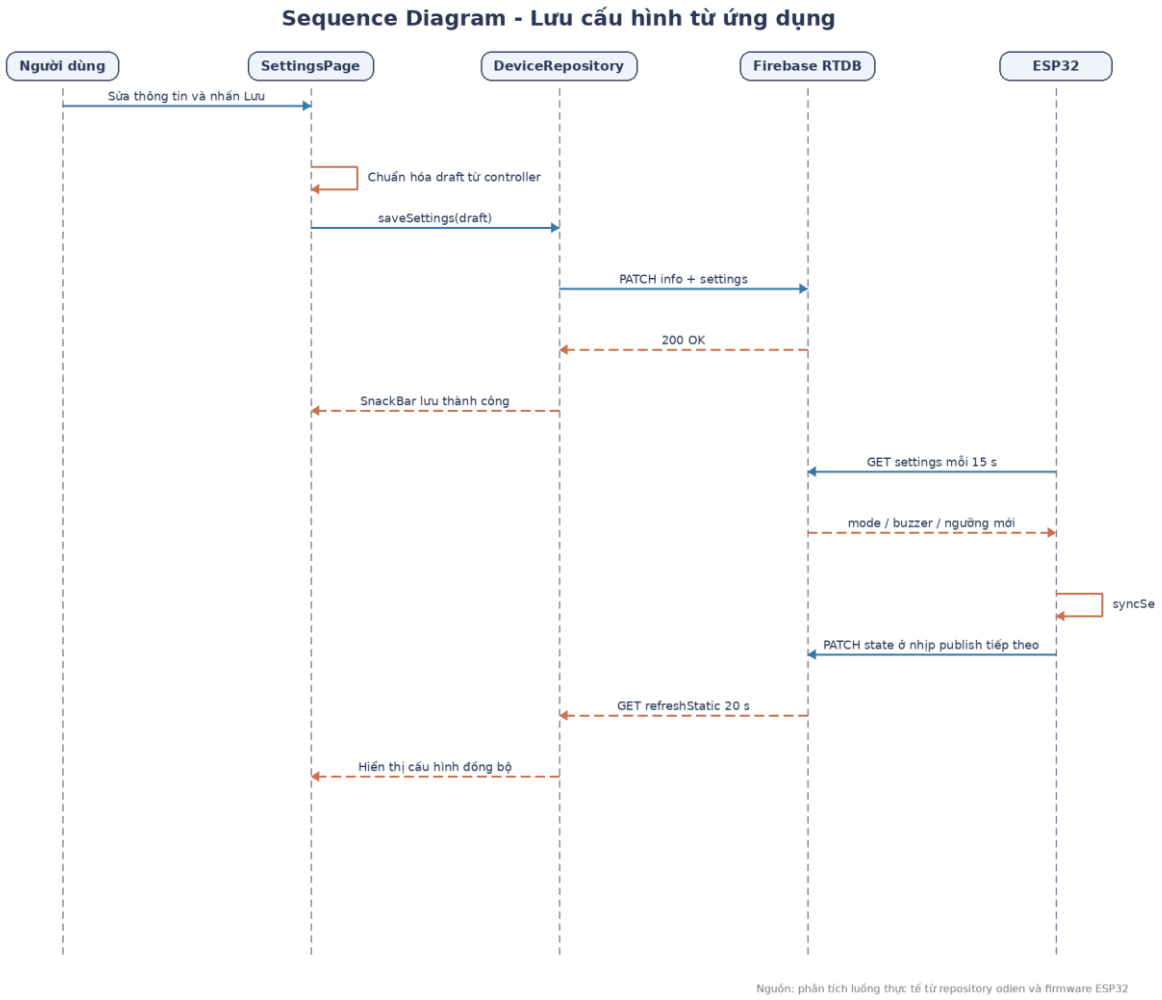
Hình 10. Sequence Diagram khởi động ứng dụng và tải dashboard.

Sequence Diagram khởi động mô tả toàn bộ vòng đời từ lúc mở app đến khi dashboard nhận được state đầu tiên. AppBootstrapPage chỉ làm một bước đệm ngắn 180 ms, sau đó chuyển sang ConnectionGatePage. Sau khi DEVICE_ID hợp lệ, watchDashboard() mới được kích hoạt. Giai đoạn bootstrap gồm cả GET dữ liệu tĩnh lẫn mở SSE tới state.



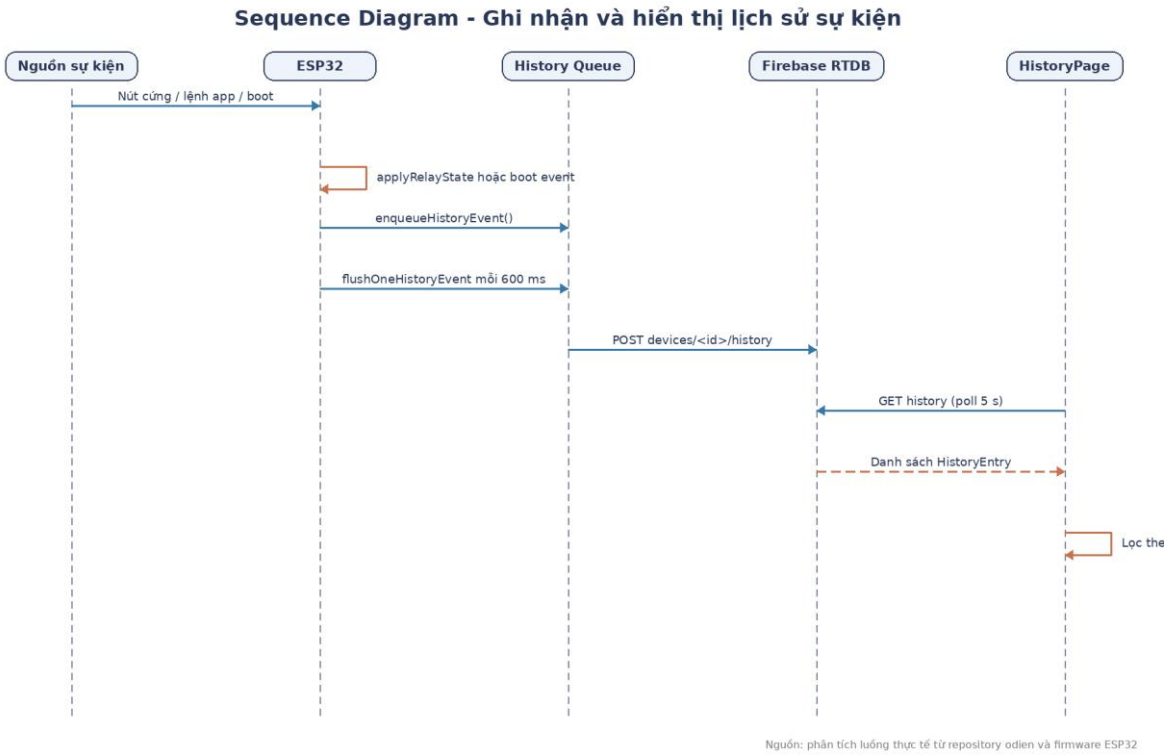
Hình 11. Sequence Diagram gửi lệnh điều khiển relay.

Sequence Diagram gửi lệnh relay cho thấy rõ vị trí của ESP32 trong chuỗi xử lý. Nếu chỉ nhìn vào app, người học rất dễ tưởng việc PUT command/latest đã tương đương với relay đổi trạng thái. Sơ đồ này chứng minh điều ngược lại: relay chỉ đổi sau khi firmware poll, kiểm tra commandId, áp lệnh, ghi ack và publish state mới.



Hình 12. Sequence Diagram lưu cấu hình từ ứng dụng.

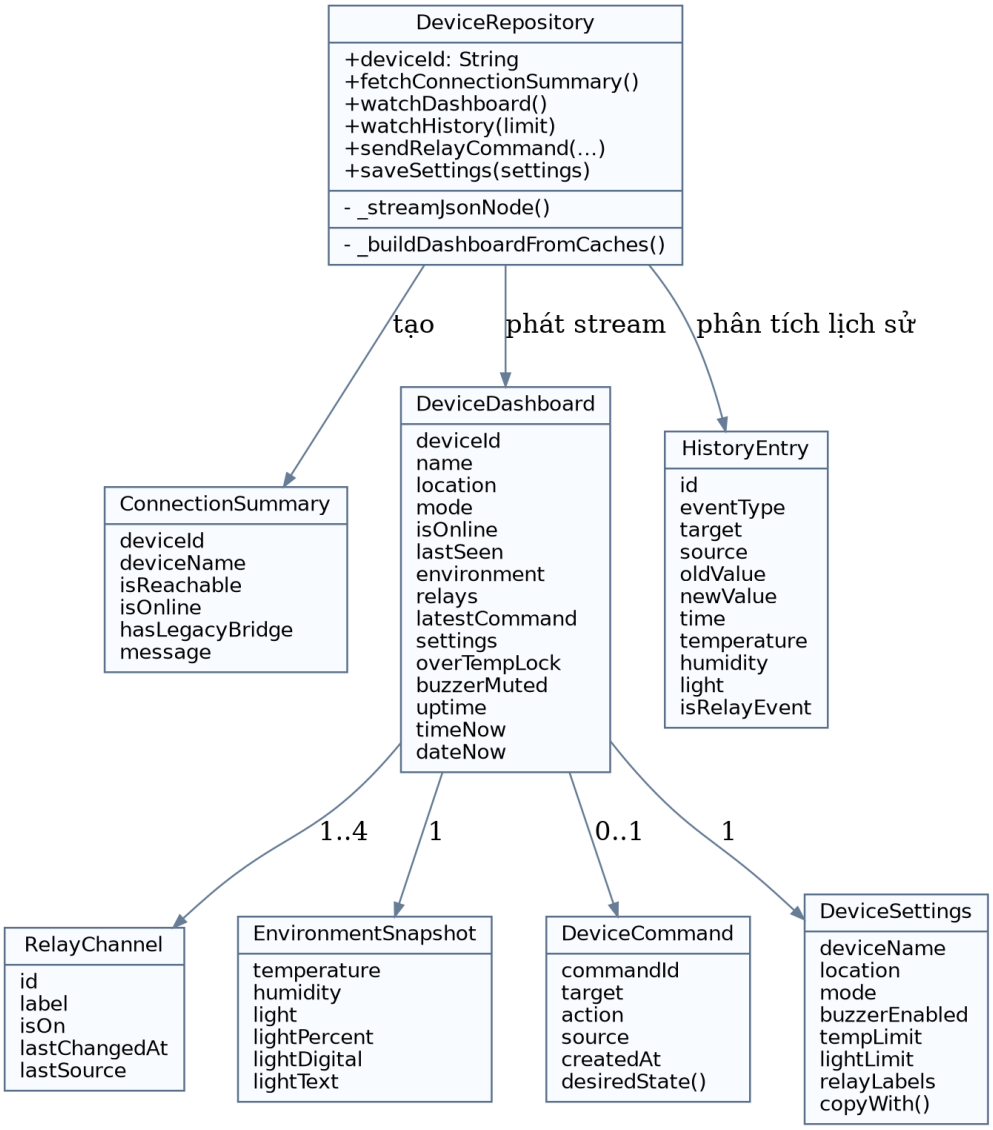
Sequence Diagram lưu cấu hình nhấn mạnh mối quan hệ giữa SettingsPage, DeviceRepository, Firebase và ESP32. Trong phiên bản hiện tại, hai thông số mode và lightLimit đã có đường đi đầy đủ qua giao diện và dữ liệu, nhưng phần logic tự động hóa tương ứng trên firmware vẫn chưa được triển khai sâu. Vì vậy, sơ đồ được dùng như một tài liệu thiết kế cho cả hiện trạng và định hướng mở rộng.



Hình 13. Sequence Diagram ghi nhận và hiển thị lịch sử sự kiện.

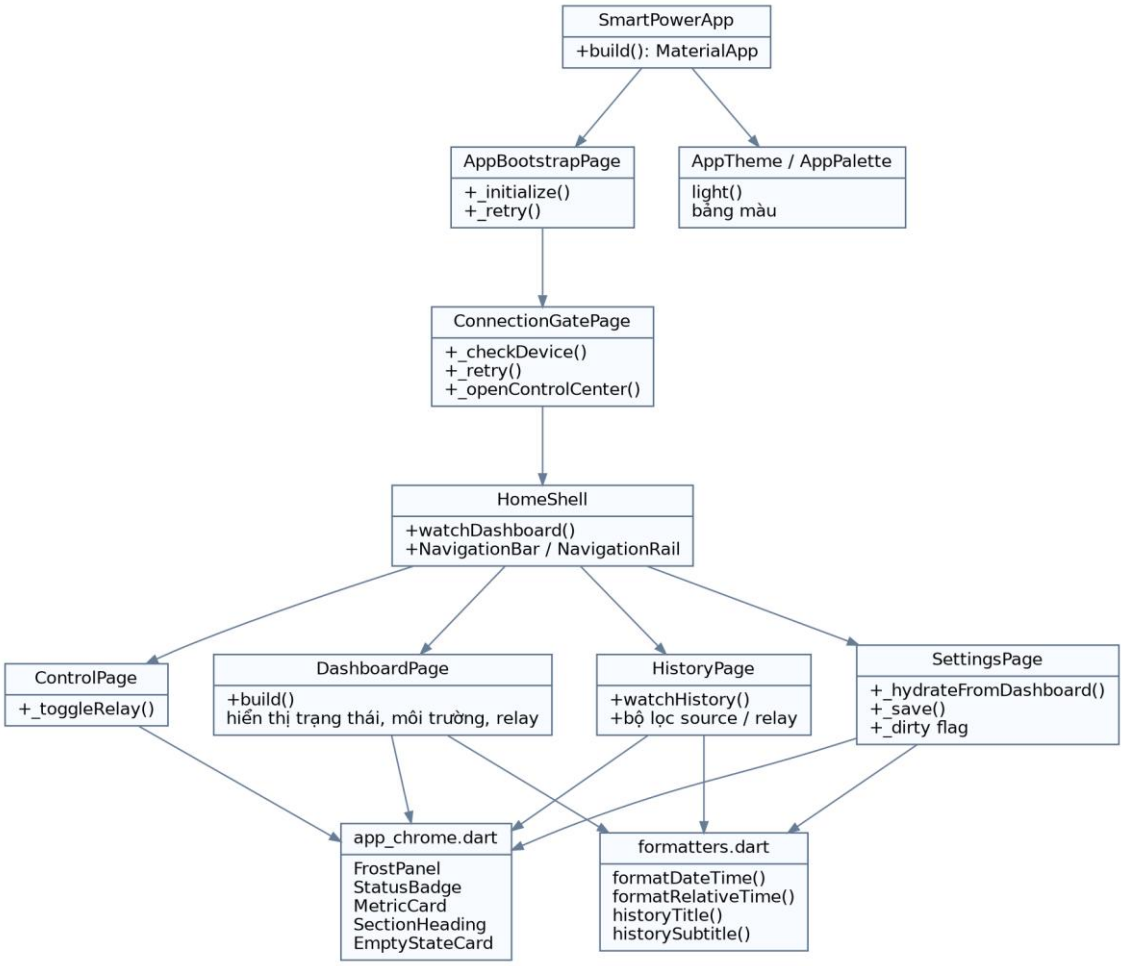
Sequence Diagram lịch sử sự kiện chứng minh rằng history không được tạo ở phía app mà được sinh ra từ nguồn sự kiện thực tế trong firmware. Khi app xem lịch sử, nó chỉ đóng vai trò đọc và lọc. Thiết kế này phù hợp với nguyên tắc dữ liệu thực phải xuất phát từ phần cứng hoặc từ bộ thực thi lệnh, tránh việc app tự ghi sự kiện không phản ánh đúng thiết bị.

3.4 Sơ đồ lớp, component, deployment và state



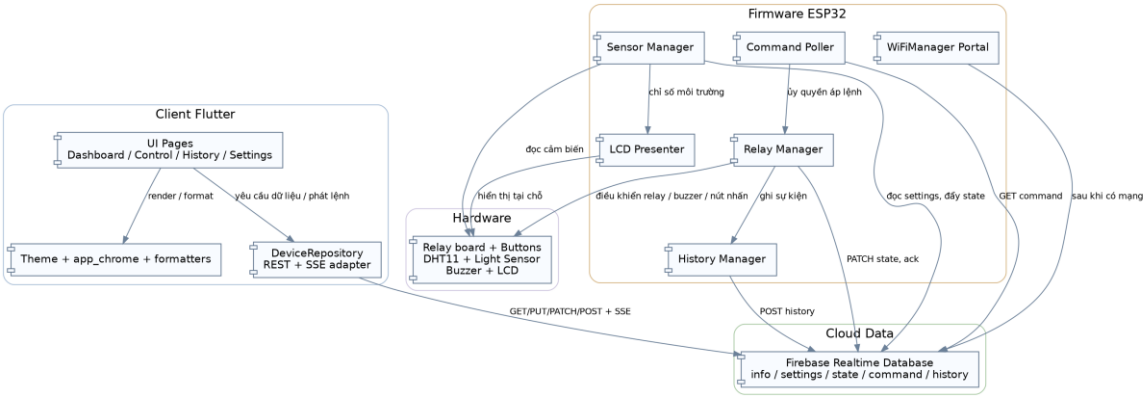
Hình 14. Class Diagram tầng mô hình và dữ liệu.

Class Diagram tầng mô hình và dữ liệu cho thấy DeviceRepository là điểm hội tụ của toàn bộ luồng parse và build dữ liệu. Từ ConnectionSummary, DeviceDashboard, DeviceCommand đến HistoryEntry và DeviceSettings, mọi đối tượng đều được gom trong device_models.dart. Đây là điểm tốt của dự án vì nó tách kiểu dữ liệu khỏi tầng giao diện.



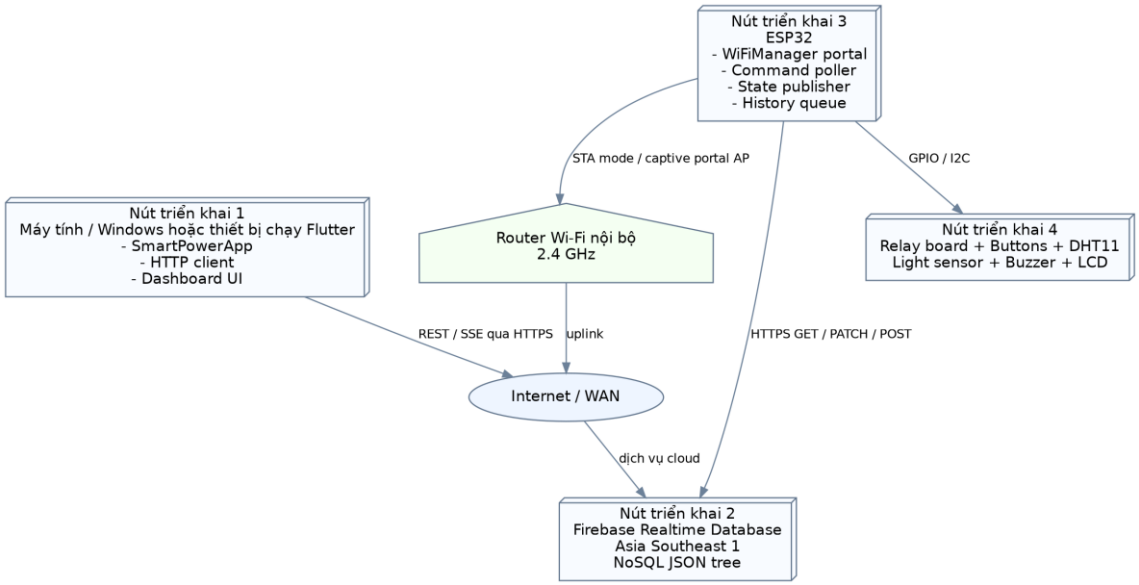
Hình 15. Class Diagram tầng giao diện và điều hướng.

Class Diagram tầng giao diện làm rõ mối quan hệ điều hướng giữa SmartPowerApp, AppBootstrapPage, ConnectionGatePage và HomeShell. Sau HomeShell là bốn màn hình chức năng. app_chrome.dart và AppTheme đóng vai trò hạ tầng giao diện dùng chung, giúp tái sử dụng panel, badge, card và màu sắc trên toàn bộ dự án.



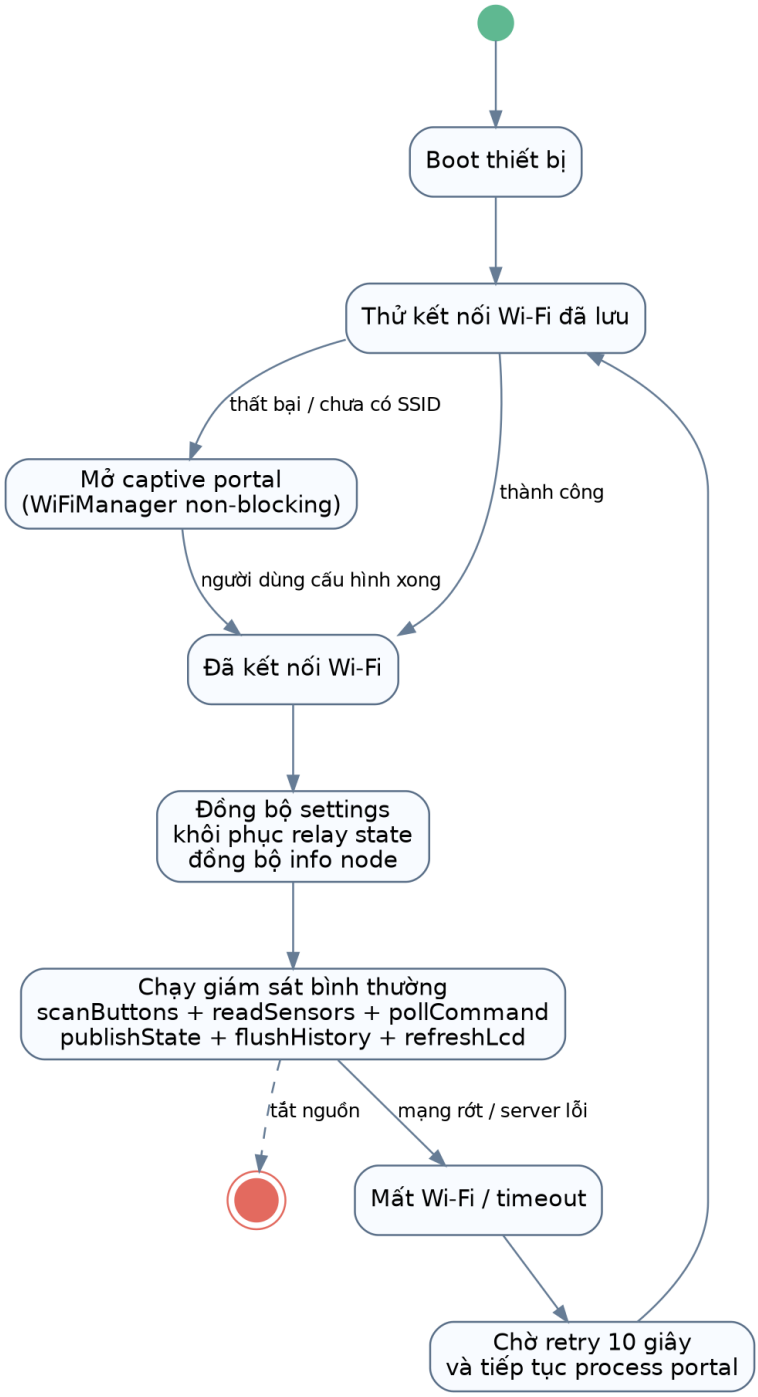
Hình 16. Component Diagram của hệ thống.

Component Diagram mô tả rõ những thành phần có thể xem như mô-đun độc lập. Ở phía client là UI pages, theme/widgets và DeviceRepository. Ở phía firmware là WiFiManager portal, command poller, sensor manager, relay manager, history manager và LCD presenter. Sơ đồ này đặc biệt hữu ích nếu sau này nhóm muốn tách project theo package hoặc tái cấu trúc firmware theo lớp.



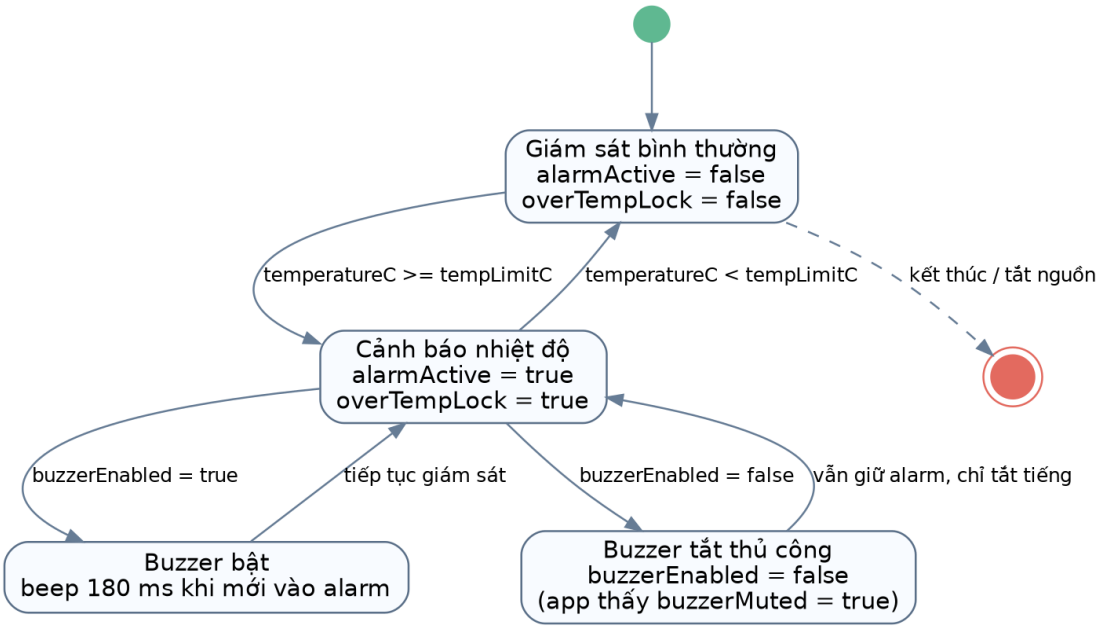
Hình 17. Deployment Diagram khi triển khai thực tế.

Deployment Diagram đặt hệ thống vào bối cảnh triển khai thật: một thiết bị chạy Flutter client, một dịch vụ cloud Firebase, một router Wi-Fi nội bộ và một nút ESP32 nối với phần cứng ngoại vi. Việc biểu diễn các nút triển khai giúp báo cáo phân biệt rõ đâu là thành phần logic, đâu là thành phần hạ tầng.



Hình 18. State Diagram trạng thái kết nối của thiết bị ESP32.

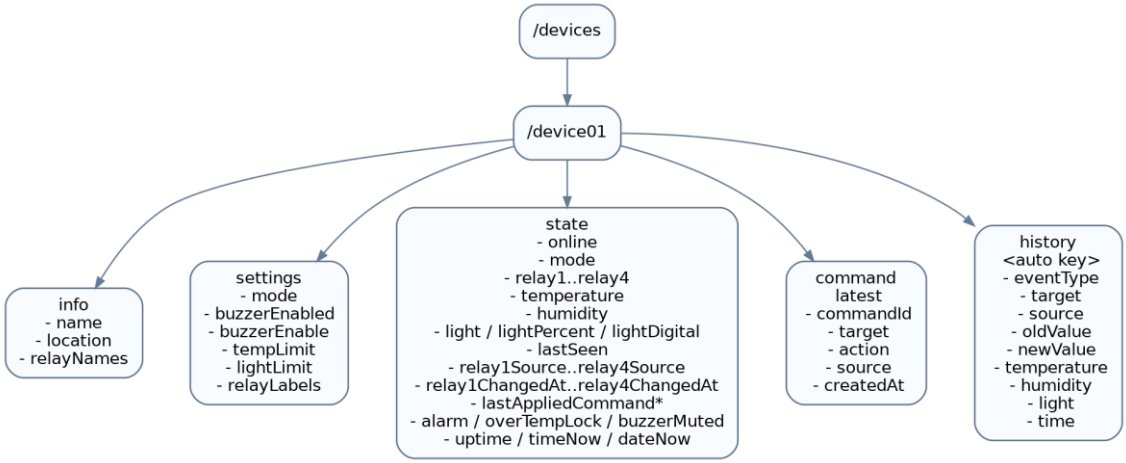
State Diagram kết nối mô tả vòng đời của thiết bị từ boot, thử kết nối Wi-Fi, mở captive portal, đồng bộ dữ liệu rồi chạy giám sát bình thường. Khi mất mạng, hệ thống quay về trạng thái retry sau 10 giây. Đây là một vòng lặp quan trọng trong firmware để duy trì tính bền bỉ của thiết bị ngoài thực tế.



Hình 19. State Diagram trạng thái cảnh báo nhiệt độ.

State Diagram cảnh báo nhiệt độ được xây dựng dựa trên logic updateAlarmState() trong firmware. Khi nhiệt độ môi trường vượt ngưỡng tempLimitC, alarmActive được bật và nếu buzzerEnabled đang true thì thiết bị beep 180 ms. Nếu người dùng đã tắt buzzer trong settings, app vẫn thấy trạng thái alarm nhưng không nghe còi cảnh báo. Đây là điểm cần diễn giải rõ để tránh hiểu nhầm rằng tắt buzzer đồng nghĩa tắt luôn logic alarm.

3.5 Thiết kế dữ liệu Firebase Realtime Database



Hình 20. Cấu trúc cây dữ liệu Firebase Realtime Database.

Cây dữ liệu Firebase được thiết kế khá mạch lạc quanh /devices/device01. Nhóm info và settings chứa dữ liệu ít thay đổi; state là trạng thái vận hành; command/latest là lệnh chờ xử lý; còn history là nhật ký sự kiện. Kiến trúc này phù hợp với yêu cầu đọc state nhanh và ghi history tuần tự theo thời gian. (Hà Nguyễn Gia Khang, 2026; Google, 2025b).

Một ưu điểm khác là dự án tách rõ dữ liệu "mô tả" và dữ liệu "vận hành". Ví dụ, name/location/relayNames nằm ở info; mode/ buzzerEnabled/ tempLimit/ lightLimit nằm ở settings; còn trạng thái relay, lastSeen, sensor values và command ack nằm ở state. Nhờ vậy, dashboard có thể bootstrap hiệu quả mà không phải suy luận lẫn lộn giữa các trường.

Bảng 14. Từ điển dữ liệu node info.

Trường	Kiểu	Ý nghĩa	Ghi chú
name	String	Tên thiết bị hiển thị trên dashboard	App và firmware cùng sử dụng
location	String	Vị trí lắp đặt của thiết bị	App và firmware cùng sử dụng
relayNames	Map<String,String>	Tên hiển thị cho các relay	Ứng dụng cập nhật, dashboard đọc

Bảng 15. Từ điển dữ liệu node settings.

Trường	Kiểu	Ý nghĩa	Ghi chú

mode	String	Chế độ manual/auto	Firmware hiện mới đồng bộ và hiển thị
buzzerEnabled	bool	Bật/tắt âm thanh phản hồi	Được app và firmware cùng đọc/ghi
buzzerEnable	bool	Biến tương thích dữ liệu cũ	Được repository vẫn hỗ trợ đọc
tempLimit	double	Ngưỡng nhiệt độ cảnh báo	Firmware dùng để bật alarm
lightLimit	double	Ngưỡng ánh sáng dự kiến	Đã có dữ liệu nhưng logic chưa dùng sâu
relayLabels	Map<String,String>	Tên các kênh relay	SettingsPage lưu, Dashboard đọc

Bảng 16. Từ điển dữ liệu node state (phần 1).

Trường	Kiểu	Ý nghĩa
online	bool	Thiết bị còn sống và có mạng
mode	String	Chế độ hiện tại của firmware
relay1..relay4	bool	Trạng thái từng relay
temperature	double	Nhiệt độ môi trường từ DHT11
humidity	double	Độ ẩm môi trường từ DHT11
light	double/int	Giá trị analog ánh sáng nếu được bật
lightPercent	double/int	Tỉ lệ ánh sáng quy đổi phần trăm
lightDigital	bool	Trạng thái digital sáng/tối
lightText	String	Nhãn hiển thị Sáng/Tối
lastSeen	timestamp	Heartbeat cuối cùng do server gán

Bảng 17. Từ điển dữ liệu node state (phần 2).

Trường	Kiểu	Ý nghĩa
relay1Source..relay4Source	String	Nguồn thay đổi relay: button, app, esp...
relay1ChangedAt..relay4ChangedAt	timestamp	Thời điểm relay đổi trạng thái

lastAppliedCommandId	String	Command cuối đã áp lệnh
lastAppliedCommandTarget	String	Đích của command đã áp
lastAppliedCommandAction	String	Hành động đã áp
lastAppliedCommandSource	String	Nguồn lệnh đã áp
lastAppliedCommandStatus	String	Trạng thái applied/noop/ignored
alarm / overTempLock	bool	Cờ cảnh báo và khóa nhiệt
buzzerMuted	bool	Cờ phản ánh app đã tắt âm thanh
uptime / timeNow / dateNow	String	Thông tin phụ để hiển thị dashboard/LCD

Bảng 18. Từ điển dữ liệu node command/latest.

Trường	Kiểu	Ý nghĩa
commandId	String	Mã lệnh duy nhất, thường dạng cmd_<timestamp>
target	String	Đích lệnh, ví dụ relay1
action	String	on/off/toggle
source	String	Nguồn phát lệnh, thường là app
createdAt	int/timestamp	Thời điểm tạo lệnh

Bảng 19. Từ điển dữ liệu node history.

Trường	Kiểu	Ý nghĩa
eventType	String	Loại sự kiện: relay_change, boot...
target	String	Đối tượng tác động, ví dụ relay2 hoặc system
source	String	Nguồn phát sinh: button, app, esp32
oldValue	bool?	Giá trị cũ nếu có
newValue	bool?	Giá trị mới nếu có

temperature / humidity / light	number?	Snapshot môi trường đi kèm
time	server timestamp	Thời điểm server ghi sự kiện

CHƯƠNG 4: KẾT QUẢ XÂY DỰNG VÀ THẢO LUẬN

4.1 Môi trường phát triển và cấu trúc repository

Về môi trường phát triển, dự án sử dụng Flutter/Dart cho lớp ứng dụng và Arduino framework cho firmware ESP32. Ở repo hiện tại có các thư mục android, web, windows, lib và test, cho thấy ứng dụng được tổ chức theo hướng đa nền tảng. Tuy nhiên, thông điệp khởi động trong AppBootstrapPage nhấn mạnh rõ bối cảnh chạy dashboard REST trên Windows, do đó có thể xem desktop là mục tiêu ưu tiên ở phiên bản đang phân tích. (Hà Nguyễn Gia Khang, 2026).

Bảng 20. Cấu trúc thư mục và mô-đun chính của repository.

Đường dẫn	Vai trò	Mô tả
lib/main.dart	Entry point tối giản	Khởi tạo WidgetsBinding và chạy SmartPowerApp
smart_power_app.dart	Khởi tạo MaterialApp	Định nghĩa AppBootstrapPage và điều hướng ban đầu
lib/screens/connection_gate_page.dart	Cổng kiểm tra thiết bị	Nhập DEVICE_ID, đọc ConnectionSummary và mở HomeShell
lib/screens/home_shell.dart	Shell điều hướng	Nghe stream DeviceDashboard và chuyển bốn tab chức năng
lib/screens/dashboard_page.dart	Tổng quan	Hiển thị online/offline, cảm biến, relay, lệnh gần nhất
lib/screens/control_page.dart	Điều khiển	Gửi lệnh bật/tắt relay qua sendRelayCommand
lib/screens/history_page.dart	Lịch sử	Đọc history và lọc theo source/relay
lib/screens/settings_page.dart	Cài đặt	Lưu tên thiết bị, mode, buzzer, ngưỡng và nhãn relay
lib/data/device_repository.dart	Tầng dữ liệu	GET/PUT/PATCH/POST + SSE, build dashboard, parse history
lib/models/device_models.dart	Kiểu dữ liệu	Định nghĩa DeviceDashboard, HistoryEntry, DeviceSettings...
lib/core/theme / utils / widgets	Hạ tầng giao diện	Theme, formatter và widget tái sử dụng

firmware/odienthongminh_esp32.ino	Thiết bị nhúng	Điều khiển phần cứng và đồng bộ Firebase

4.2 Hiện thực ứng dụng Flutter

Ứng dụng Flutter được tổ chức với điểm vào gọn tại lib/main.dart, sau đó chuyển sang SmartPowerApp để khởi tạo MaterialApp, theme và luồng điều hướng chính. Cách tổ chức này giúp tách phần khởi động ứng dụng khỏi các màn hình chức năng.

ConnectionGatePage được dùng như lớp xác định thiết bị đích trước khi truy cập dashboard. Màn hình này yêu cầu nhập DEVICE_ID, tạo DeviceRepository theo định danh đã nhập và gọi hàm kiểm tra dữ liệu info, state và settings để xác định node thiết bị có tồn tại hay không.

HomeShell đóng vai trò shell điều hướng của hệ thống. StreamBuilder tại đây lắng nghe watchDashboard() để nhận DeviceDashboard và cung cấp cùng một nguồn dữ liệu cho bốn màn hình DashboardPage, ControlPage, HistoryPage và SettingsPage.

DashboardPage hiển thị các thông tin trạng thái tổng quan của thiết bị như online/offline, mode, buzzer, alarm, dữ liệu môi trường, trạng thái relay và lệnh đã áp gần nhất. Cấu trúc này phù hợp với yêu cầu quan sát nhanh hệ thống tại một màn hình trung tâm.

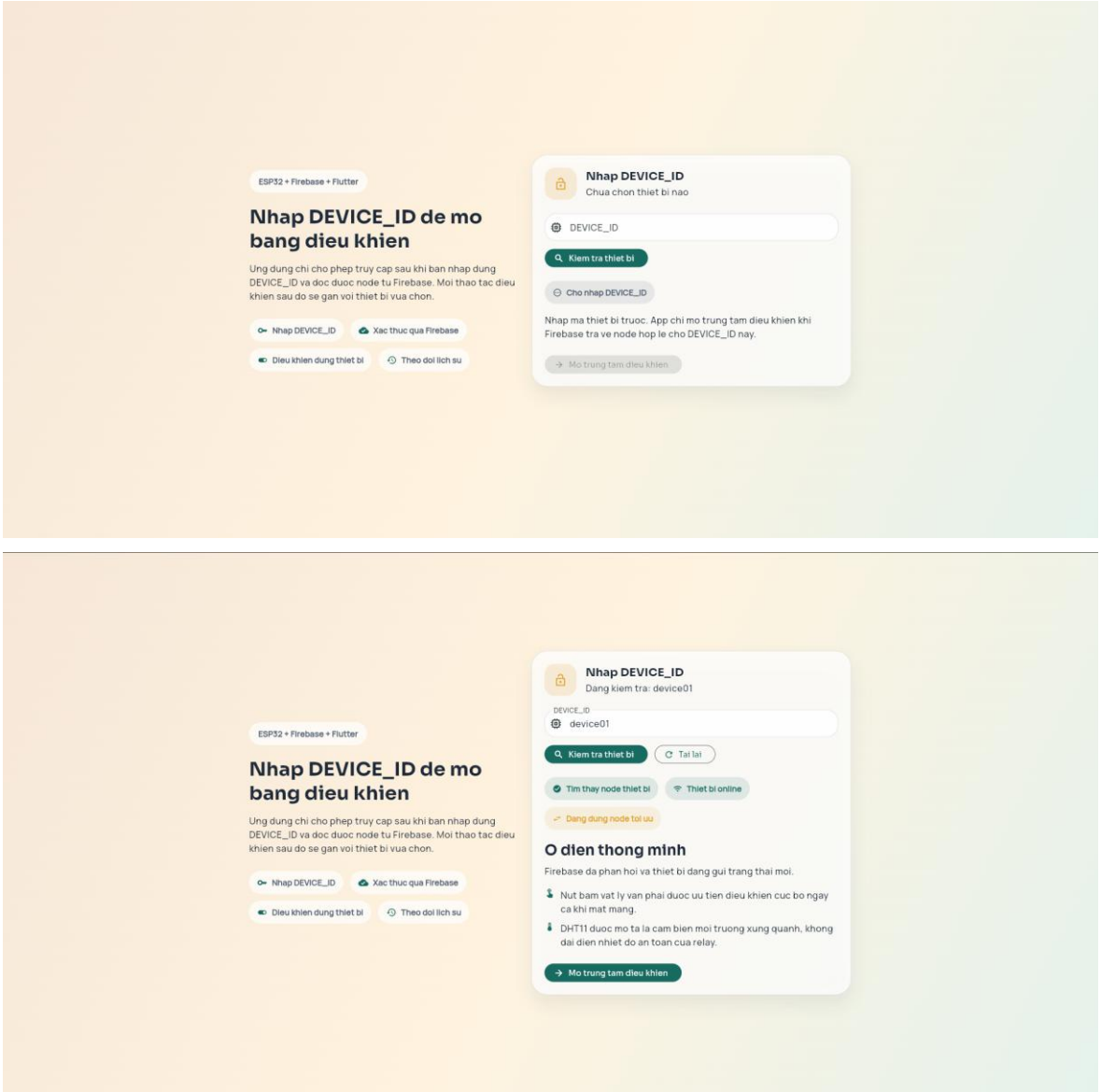
ControlPage được tổ chức theo từng relay với nhãn, trạng thái, nguồn thay đổi, thời điểm cập nhật và công tắc điều khiển. Khi có thao tác bật/tắt, màn hình gọi repository.sendRelayCommand() để ghi lệnh lên command/latest thay vì thay đổi trực tiếp mô hình dữ liệu trên giao diện.

HistoryPage sử dụng watchHistory() để đọc node history theo chu kỳ và parse dữ liệu thành HistoryEntry. Mô hình dữ liệu lịch sử không chỉ lưu loại sự kiện và nguồn kích hoạt, mà còn lưu snapshot nhiệt độ, độ ẩm và ánh sáng đi kèm từng mốc thời gian.

Ngoài các màn hình chính, app_theme.dart và app_chrome.dart cung cấp các thành phần giao diện dùng chung như panel, badge trạng thái, thẻ chỉ số và tiêu đề khu vực. Nhờ đó, toàn bộ dashboard có một hệ thống trình bày thống nhất hơn ở mức component.

4.2.1 Giao diện nhập DEVICE_ID

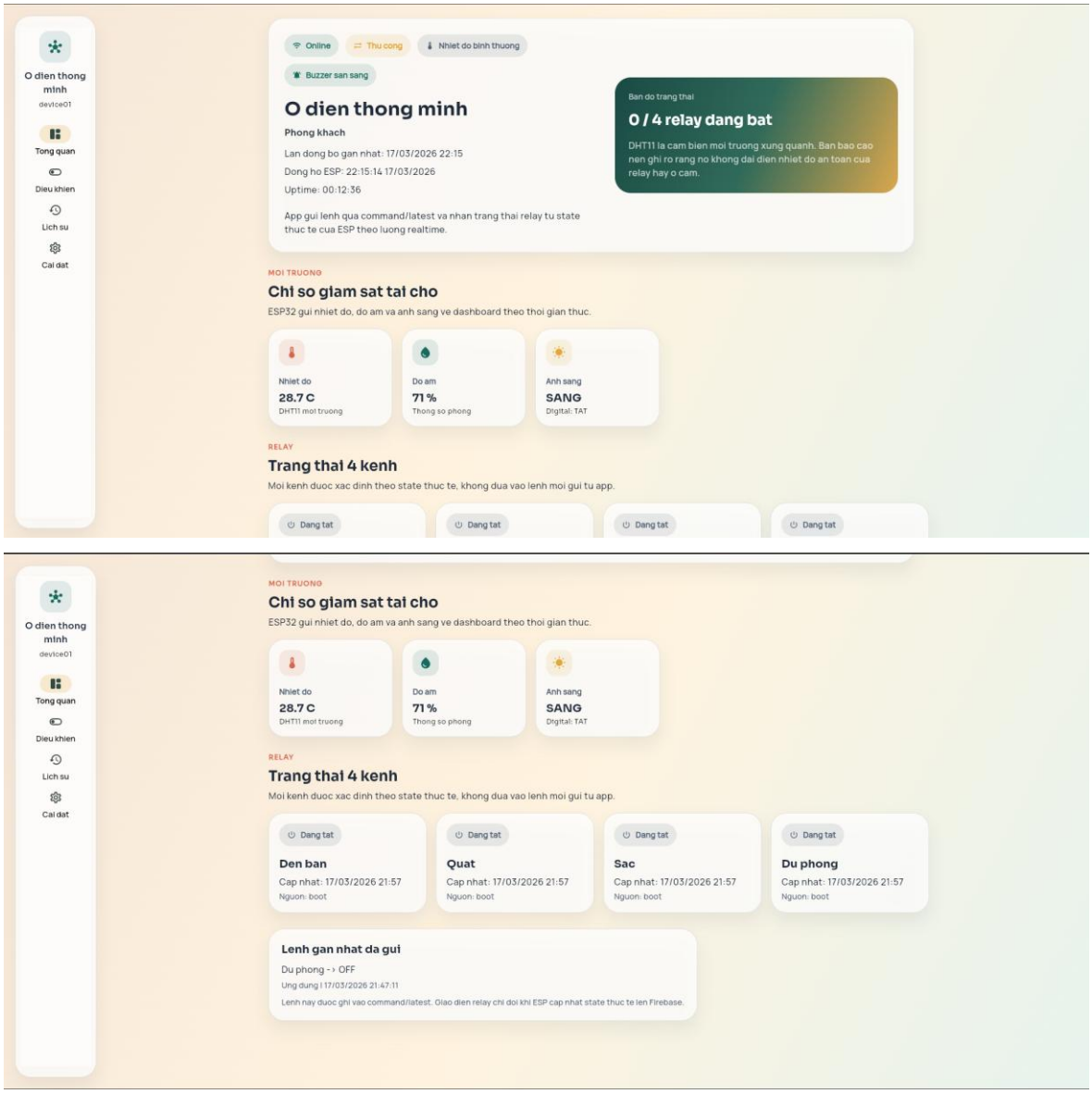
Màn hình nhập DEVICE_ID đảm nhiệm chức năng xác định thiết bị đích trước khi hệ thống khởi tạo HomeShell. Ảnh chèn tại vị trí này nên thể hiện ô nhập định danh, nút kiểm tra, thông báo trạng thái kết nối và thông điệp lỗi khi node thiết bị không tồn tại.



Hình 21. Giao diện nhập DEVICE_ID.

4.2.2 Giao diện dashboard tổng quan

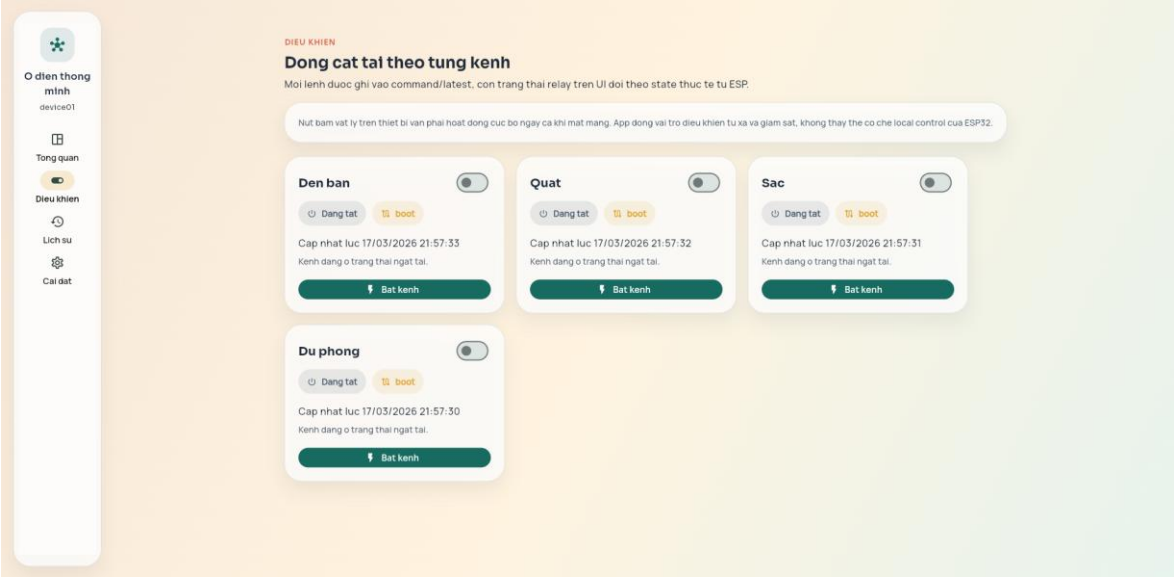
Ảnh dashboard nên thể hiện các vùng thông tin online/offline, trạng thái mode, buzzer, alarm, số liệu môi trường và trạng thái bốn relay. Đây là hình minh chứng trực quan cho vai trò của DashboardPage trong việc tổng hợp state thực tế từ thiết bị.



Hình 22. Giao diện dashboard tổng quan.

4.2.3 Giao diện điều khiển relay

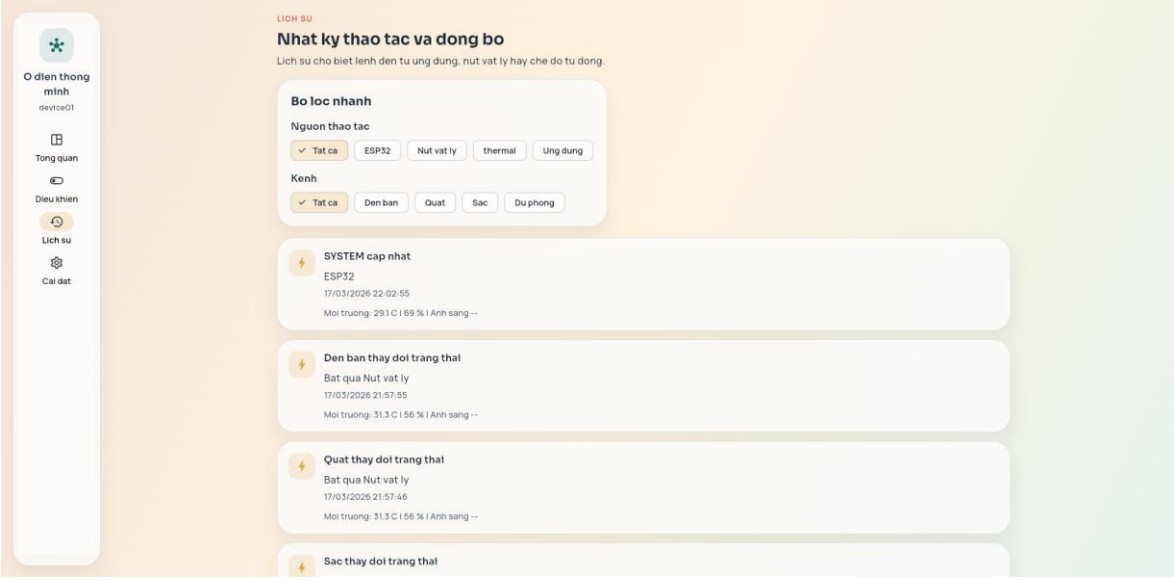
Ảnh điều khiển relay nên thể hiện rõ các công tắc bật/tắt, nhãn relay, nguồn kích hoạt và thời điểm cập nhật gần nhất. Hình này giúp minh họa luồng gửi lệnh từ ControlPage lên command/latest và cách giao diện chờ state phản hồi từ ESP32.



Hình 23. Giao diện điều khiển relay.

4.2.4 Giao diện lịch sử hoạt động

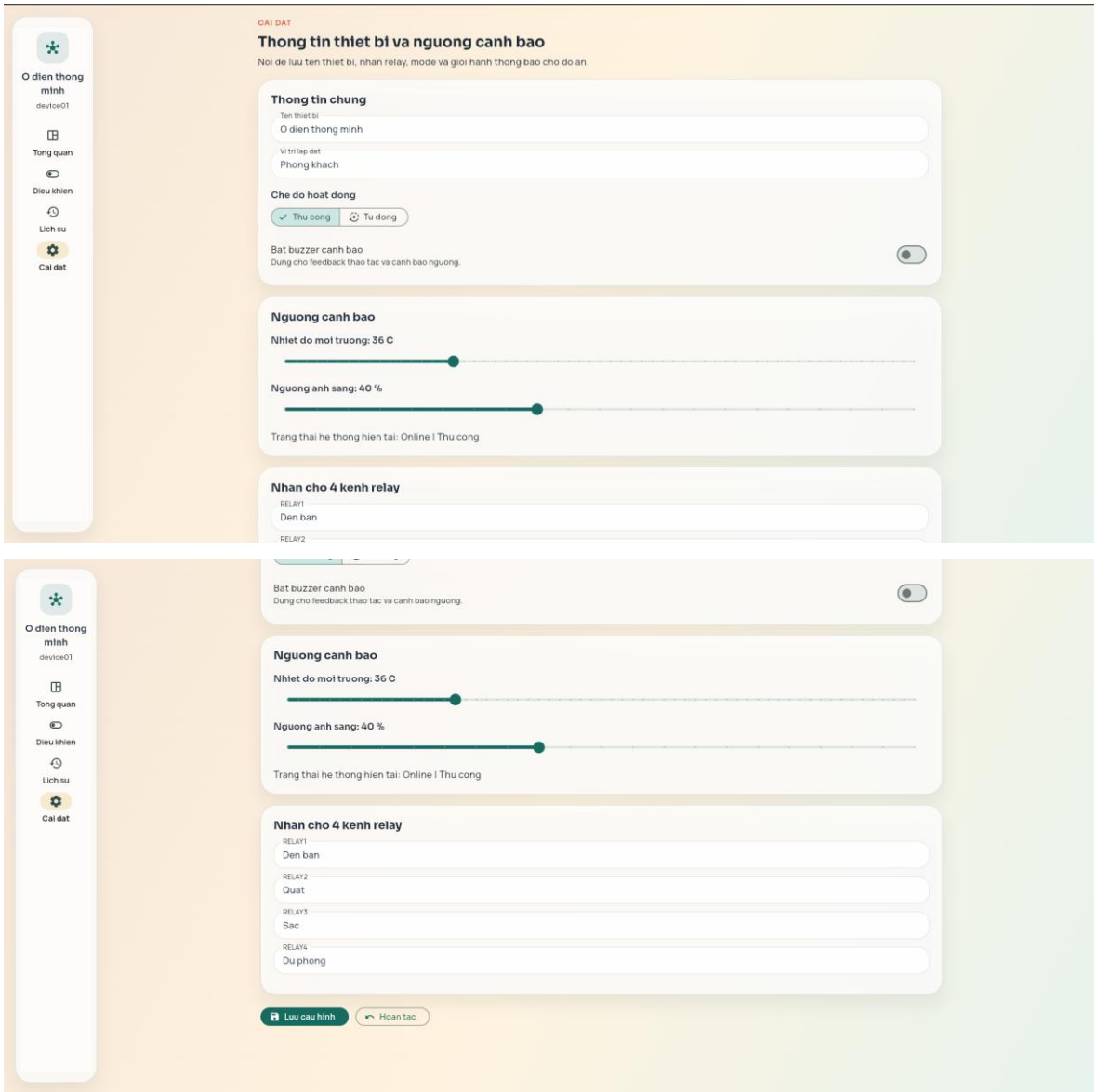
Ảnh màn hình lịch sử cần thể hiện danh sách sự kiện, bộ lọc theo source hoặc relay và thông tin ngữ cảnh đi kèm từng entry. Hình này hỗ trợ mô tả chức năng giám sát hậu kiểm của HistoryPage.



Hình 24. Giao diện lịch sử hoạt động.

4.2.5 Giao diện cài đặt thiết bị

Ảnh cài đặt nên thể hiện các trường tên thiết bị, vị trí, mode, buzzer, ngưỡng cảnh báo và nhãn relay. Tại vị trí này nên chụp cả trạng thái chưa lưu và sau khi lưu để làm rõ cơ chế _dirty flag và đồng bộ settings.



Hình 25. Giao diện cài đặt thiết bị.

4.3 Hiện thực firmware ESP32

Firmware là lớp thực thi trực tiếp các thao tác điều khiển và đồng bộ dữ liệu của hệ thống. Ở đầu tệp .ino, các hằng số cấu hình được khai báo theo nhóm gồm tên AP, đường dẫn Firebase, DEVICE_ID, ánh xạ GPIO, logic active-low/active-high và các chu kỳ tác vụ.

Firmware sử dụng WiFiManager ở chế độ non-blocking, nghĩa là portal cấu hình có thể được xử lý trong vòng lặp chính mà không khóa toàn bộ chương trình. Cách triển khai này phù hợp với yêu cầu vừa cấu hình mạng vừa duy trì các tác vụ định kỳ khác của thiết bị.

Khối điều khiển relay được xây quanh hàm applyRelayState(). Hàm này đồng thời xử lý thay đổi trạng thái relay phần cứng, cập nhật nguồn kích hoạt, đánh dấu cờ dirty cho state, ghi lịch sử khi cần và kích hoạt phản hồi âm thanh tùy cấu hình.

Khối lệnh cloud được tổ chức trong pollFirebaseCommand(). Theo chu kỳ 500 ms, firmware đọc command/latest, loại bỏ commandId cũ, kiểm tra target và action, sau đó áp lệnh hoặc ghi nhận trạng thái ignored/noop rồi publish state mới lên Firebase.

Khối cảm biến và cảnh báo trong firmware bao gồm readSensors() để cập nhật nhiệt độ, độ ẩm và trạng thái sáng/tối; đồng thời updateAlarmState() so sánh temperatureC với tempLimitC để kích hoạt cờ alarm và phản hồi buzzer khi vượt ngưỡng.

Khối publish state chỉ đẩy dữ liệu khi có thay đổi hoặc tới chu kỳ định sẵn, qua đó hạn chế số lần ghi không cần thiết lên Firebase. Payload state được tổ chức tập trung để dashboard có thể lấy đầy đủ relay, cảm biến, command ack và thông tin hiển thị phụ trợ từ một node.

Khối history queue sử dụng hàng đợi vòng tròn để ghi sự kiện theo nhịp, trong khi LCD luân phiên nhiều trang hiển thị nhằm thể hiện trạng thái Wi-Fi, relay, nhiệt độ, độ ẩm và ánh sáng. Hai khối này mở rộng thiết bị từ mức điều khiển cơ bản sang mức có ghi nhận vận hành và quan sát tại chỗ.

Bảng 21. Bảng ánh xạ chân ESP32 và ngoại vi.

GPIO	Ngoại vi	Vai trò
GPIO4	DHT11 DATA	Đọc nhiệt độ và độ ẩm môi trường
GPIO14	Light DO	Đọc sáng/tối ở ngõ digital
GPIO27	Light AO	Ngõ analog dự phòng, hiện tắt mặc định
GPIO16	Relay 1	Điều khiển tải 1 (Đèn bàn)
GPIO17	Relay 2	Điều khiển tải 2 (Quạt)
GPIO18	Relay 3	Điều khiển tải 3 (Sạc)
GPIO19	Relay 4	Điều khiển tải 4 (Dự phòng)
GPIO33	Button 1	Nút vật lý số 1, cần nhấn giữ 220 ms
GPIO32	Button 2	Nút vật lý số 2, xử lý ngay sau debounce

GPIO35	Button 3	Nút vật lý số 3, cần pull-up ngoài
GPIO34	Button 4	Nút vật lý số 4, cần pull-up ngoài
GPIO26	Buzzer	Âm thanh phản hồi và cảnh báo
GPIO21/22	LCD I2C	SDA/SCL cho màn hình 16x2

Bảng 22. Bảng timer và chu kỳ tác vụ trong firmware.

Hằng số	Giá trị	Ý nghĩa
BUTTON_DEBOUNCE_MS	55 ms	Lọc nhiễu nút nhấn
BUTTON1_SOFT_HOLD_MS	220 ms	Yêu cầu giữ mềm cho nút 1
WIFI_RETRY_MS	10000 ms	Chu kỳ thử nối lại Wi-Fi
SENSOR_READ_MS	3000 ms	Nhịp đọc DHT11 và cảm biến sáng
COMMAND_POLL_MS	500 ms	Nhịp đọc command/latest
SETTINGS_POLL_MS	15000 ms	Nhịp đồng bộ settings từ Firebase
STATE_PUSH_MS	5000 ms	Nhịp đẩy state định kỳ
HISTORY_FLUSH_MS	600 ms	Nhịp POST một sự kiện lịch sử
LCD_REFRESH_MS	1000 ms	Nhịp refresh LCD
LCD_PAGE_MS	2500 ms	Nhịp chuyển trang LCD

4.4 Luồng dữ liệu đầu cuối và thảo luận kỹ thuật

Nếu nhìn toàn cục, hệ thống vận hành theo mô hình request - execute - confirm. Người dùng gửi yêu cầu từ ứng dụng; cloud đảm nhiệm vai trò trung gian lưu trữ và chuyển tiếp; ESP32 thực thi lệnh; sau đó trạng thái xác nhận được trả về qua node state.

Thiết kế hiện tại cũng có các giới hạn kỹ thuật cần ghi nhận. Do command/latest mới là một node đơn, hai nguồn lệnh phát gần thời điểm nhau có thể dẫn đến khả năng lệnh trước bị ghi đè trước khi firmware kịp poll.

Repository sử dụng SSE cho node state và polling cho history, info, settings, command. Cách kết hợp push và poll này phù hợp với yêu cầu thời gian thực của

trạng thái vận hành nhưng vẫn giữ mức độ triển khai đơn giản đối với các node ít thay đổi hơn.

Trong phiên bản hiện tại, mode auto và lightLimit đã có mặt trong giao diện, model, settings và luồng đồng bộ firmware, nhưng chưa được hiện thực thành luật điều khiển tự động hoàn chỉnh. Vì vậy, các thành phần này cần được mô tả như phần cấu hình đã chuẩn bị sẵn cho mở rộng, không xem là chức năng tự động hóa hoàn tất.

CHƯƠNG 5: ĐÁNH GIÁ KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN

5.1 Kịch bản kiểm thử

Do báo cáo này được biên soạn chủ yếu trên cơ sở phân tích mã nguồn và cấu trúc hệ thống, bảng kiểm thử dưới đây được trình bày theo hướng "kịch bản chuẩn cần kiểm" kết hợp với mức độ có thể xác nhận trực tiếp từ code. Những trường hợp liên quan luồng app, repository, parse dữ liệu và đường đi command/state có thể xác nhận rất rõ từ mã nguồn. Những trường hợp liên quan hành vi phần cứng thực như relay thật, buzzer thật, cảm biến thật hoặc portal Wi-Fi trong điều kiện thực địa cần được kiểm thử bổ sung trên mô hình vật lý.

Bảng 23. Kịch bản kiểm thử và cách đánh giá.

Mã	Tình huống	Điều kiện	Kỳ vọng	Trạng thái đánh giá
TC01	Nhập DEVICE_ID hợp lệ	Có node info/state/settings	ConnectionSummary reachable=true, cho phép mở HomeShell	Xác nhận từ code
TC02	Nhập DEVICE_ID không tồn tại	Không có node dữ liệu	Thông báo chưa tìm thấy node, không mở control center	Xác nhận từ code
TC03	Thiết bị reachable nhưng lastSeen cũ	state tồn tại nhưng heartbeat cũ	App vẫn cho mở HomeShell và hiển thị offline	Xác nhận từ code
TC04	Bật relay từ ControlPage	command/latest nhận lệnh hợp lệ	ESP32 áp lệnh, ghi history, publish state mới	Cần đo kiểm phần cứng
TC05	Gửi command target sai	target không thuộc relay1..relay4	Firmware ghi ack ignored_target	Xác nhận từ code
TC06	Nhấn nút vật lý relay	Thiết bị đang chạy	Relay đổi trạng thái cục bộ và app đồng bộ sau khi có mạng	Cần đo kiểm phần cứng
TC07	Lưu settings từ SettingsPage	Dữ liệu hợp lệ	PATCH info/settings thành công, _dirty về false	Xác nhận từ code
TC08	ESP32 đọc settings mới	Có thay đổi trên settings	Firmware cập nhật mode, buzzerEnabled, tempLimit, lightLimit	Xác nhận từ code
TC09	Nhiệt độ vượt ngưỡng	temperatureC >= tempLimitC	alarmActive=true, overTempLock=true, beep nếu buzzer bật	Cần đo kiểm phần cứng

TC10	HistoryPage lọc theo source	Có nhiều nguồn sự kiện	Danh sách chỉ giữ các entry phù hợp	Xác nhận từ code
TC11	Đứt SSE tạm thời	Mạng chập chờn	Repository tự thử nối lại sau 2 giây	Xác nhận từ code
TC12	Portal Wi-Fi khi mất mạng	Thiết bị không vào được Wi-Fi đã lưu	WiFiManager mở AP và cho cấu hình lại	Cần đo kiểm phần cứng

5.2 Kết quả đạt được và giới hạn hiện tại

Xét trên góc độ kiến trúc, hệ thống đã hình thành được các thành phần nền tảng gồm luồng command/state tách bạch, local control bằng nút nhấn, tầng repository riêng cho ứng dụng Flutter và cấu trúc lưu trữ dữ liệu đủ rõ trên Firebase. Các thành phần này tạo cơ sở cho việc mở rộng thêm bảo mật, tự động hóa và giám sát nâng cao ở các giai đoạn sau.

Ở phía firmware, hệ thống không chỉ dừng ở thao tác bật/tắt relay mà còn có thêm WiFiManager portal, history queue, command ack và LCD đa trang. Điều này cho thấy thiết bị đã được tổ chức theo hướng có vận hành thực tế, không chỉ là mô hình điều khiển thử nghiệm.

Bên cạnh các thành phần đã hiện thực, hệ thống vẫn còn các giới hạn cần được nêu rõ. Các giới hạn này tập trung ở ba nhóm chính: bảo mật cloud, xử lý điều khiển đồng thời và mức độ hoàn thiện của auto mode.

Bảng 24. Phân loại hạn chế hiện tại và đề xuất cải tiến.

Nhóm hạn chế	Mô tả hiện trạng	Hướng cải tiến
Bảo mật cloud	DATABASE_AUTH đang để rỗng; chưa thấy lớp xác thực người dùng trong app	Bổ sung Firebase Auth, rules theo deviceId và token ký lệnh
Điều khiển đồng thời	command/latest có nguy cơ bị ghi đè nếu nhiều nguồn phát lệnh quá nhanh	Thiết kế hàng đợi command hoặc dùng broker/MQTT
Tự động hóa	mode auto và lightLimit chưa có luật điều khiển relay tương ứng	Bổ sung rule engine hoặc scheduler
An toàn cảm biến	DHT11 không phải cảm biến nhiệt tiếp điểm relay	Thêm cảm biến nhiệt chuyên dụng gắn gần tải hoặc relay

Giám sát offline	HistoryPage dùng polling, chưa có cơ chế cache cục bộ	Bổ sung cache local / offline-first
Bảo trì firmware	Firmware hiện là một file .ino dài hơn một nghìn dòng	Tách thành nhiều module hoặc lớp nhỏ hơn

5.3 Hướng phát triển trong tương lai

Trong giai đoạn tiếp theo, hướng phát triển hợp lý nhất là hoàn thiện lớp bảo mật. Khi đó, mỗi thiết bị nên có mã định danh riêng, rules Firebase chỉ cho phép người dùng tương ứng đọc/ghi đúng node của mình, và các lệnh điều khiển nên kèm theo thông tin xác thực rõ ràng. Đây là điều kiện gần như bắt buộc nếu muốn đưa hệ thống ra khỏi phạm vi demo học thuật.

Hướng thứ hai là hiện thực chế độ tự động đúng nghĩa. Hiện tại mode auto mới dừng ở mức dữ liệu. Trong tương lai, hệ thống có thể dùng lightLimit để tự bật/tắt đèn theo sáng tối, dùng scheduler để lập lịch cho quạt hoặc sạc, và thậm chí thêm các luật kết hợp nhiều tín hiệu môi trường. Khi đó, sequence diagram và state diagram trong báo cáo hiện tại sẽ là nền tảng rất hữu ích để mở rộng.

Hướng thứ ba là nâng cấp hạ tầng đồng bộ. Với một thiết bị, REST + SSE là đủ. Nhưng khi số lượng thiết bị tăng lên hoặc cần phản hồi nhanh hơn, nhóm có thể cân nhắc MQTT, WebSocket hoặc một message bus có cơ chế hàng đợi chuẩn hơn. Đồng thời, app nên có local cache để người dùng vẫn xem được trạng thái gần nhất ngay cả khi cloud mất kết nối tạm thời.

Cuối cùng, về phần cứng, có thể bổ sung cảm biến dòng điện hoặc công suất để hệ thống chuyển từ mô hình “ổ điện điều khiển được” sang mô hình “ổ điện giám sát năng lượng”. Khi kết hợp với lịch sử, cảnh báo và tự động hóa, sản phẩm sẽ tiến gần hơn tới một nền tảng smart plug hoàn chỉnh.

KẾT LUẬN

Báo cáo đã hệ thống hóa dự án odien theo hướng phân tích cấu trúc kỹ thuật của ứng dụng Flutter, firmware ESP32 và mô hình dữ liệu Firebase. Nội dung được tổ chức theo trình tự từ xác định yêu cầu, cơ sở lý luận, phân tích - thiết kế, hiện thực hệ thống đến kiểm thử và hướng phát triển.

Kết quả phân tích cho thấy đề tài đã hình thành được nền tảng kiến trúc cho một hệ thống ổ điện thông minh quy mô nhỏ, trong đó command và state được tách riêng, local control được duy trì, giao diện được tổ chức theo mô-đun và firmware có thêm các thành phần hỗ trợ cấu hình, lịch sử và hiển thị tại chỗ.

Với bộ sơ đồ và tài liệu đã biên soạn, dự án có thể tiếp tục được mở rộng theo các hướng như xác thực người dùng, tự động hóa theo luật, giám sát năng lượng, quản lý nhiều thiết bị hoặc bổ sung hạ tầng truyền thông thời gian thực ở mức cao hơn.

TÀI LIỆU THAM KHẢO

[1] Hà Nguyễn Gia Khang (2026), odien. GitHub repository, truy cập ngày 22/03/2026, tại https://github.com/giakhang18102004/odien.

[2] Google (2025a), Flutter documentation. Docs Flutter, truy cập ngày 22/03/2026, tại https://docs.flutter.dev/.

[3] Google (2025b), Firebase Realtime Database. Firebase Documentation, truy cập ngày 22/03/2026, tại https://firebase.google.com/docs/database.

[4] Google (2025c), Installation & Setup for REST API - Firebase Realtime Database. Firebase Documentation, truy cập ngày 22/03/2026, tại https://firebase.google.com/docs/database/rest/start.

[5] Espressif Systems (2025), ESP-IDF Programming Guide - ESP32. Espressif Documentation, truy cập ngày 22/03/2026, tại https://docs.espressif.com/projects/esp-idf/en/latest/esp32/.

[6] tzapu (2025), WiFiManager. GitHub repository, truy cập ngày 22/03/2026, tại https://github.com/tzapu/WiFiManager.

[7] ASAIR (2025), DHT11 Product Manual. Aosong/ASAIR, truy cập ngày 22/03/2026, tại https://www.aosong.com/.

PHỤ LỤC

Phụ lục A. Mẫu dữ liệu JSON minh họa

Phụ lục này đưa ra một mẫu JSON rút gọn cho device01 nhằm giúp người đọc hình dung nhanh cách tổ chức dữ liệu thực trên Firebase. Mẫu dưới đây không phải toàn bộ payload sinh ra trong mọi thời điểm, nhưng phản ánh đúng tinh thần thiết kế của repo hiện tại: tách info, settings, state, command/latest và history thành các nhánh riêng.

Bảng 25. Mẫu dữ liệu JSON minh họa cho devices/device01.

Node	Mẫu dữ liệu
devices/device01/info	{ name, location, relayNames }
devices/device01/settings	{ mode, buzzerEnabled, tempLimit, lightLimit, relayLabels }
devices/device01/state	{ online, relay1..4, temperature, humidity, lightDigital, lastSeen, relay1Source..4Source, alarm, overTempLock, ... }
devices/device01/command/latest	{ commandId, target, action, source, createdAt }
devices/device01/history/<autoKey>	{ eventType, target, source, oldValue, newValue, time, temperature, humidity, light }

Mẫu JSON gợi ý dưới đây thể hiện đầy đủ hơn cấu trúc dữ liệu dành cho một thiết bị cụ thể. Khối dữ liệu này có thể được đưa vào phụ lục để người đọc đối chiếu với từ điển dữ liệu ở Chương 3.

```
{
  "devices": {
    "device01": {
      "info": {
        "name": "O dien ban hoc",
        "location": "Phong lam viec",
        "relayNames": {
          "relay1": "Den ban",
          "relay2": "Quat",
          "relay3": "Sac",
```

```
        "relay4": "Du phong"
      }
    },
    "settings": {
      "mode": "manual",
      "buzzerEnabled": true,
      "tempLimit": 35.0,
      "lightLimit": 60.0
    },
    "state": {
      "online": true,
      "relay1": false,
      "relay2": true,
      "relay3": false,
      "relay4": false,
      "temperature": 29.5,
      "humidity": 73.0,
      "lightDigital": true,
      "alarm": false
    },
    "command": {
      "latest": {
        "commandId": "cmd_1711111111111",
        "target": "relay2",
        "action": "on",
        "source": "app"
      }
    }
  }
 }
}
```

Phụ lục B. Ghi chú triển khai và lưu ý an toàn

Khi triển khai thực tế, cần đặc biệt chú ý đến cách đấu nối tải AC, chất lượng module relay, công suất nguồn và bảo vệ cách điện. Báo cáo này chỉ mô tả kiến trúc và logic hệ thống, không thay thế cho quy trình an toàn điện. Nếu muốn thương mại hóa hoặc sử dụng lâu dài, hệ thống cần có vỏ bảo vệ, cầu chì, chống quá dòng và kiểm định phần cứng phù hợp.

Ngoài ra, nếu tiếp tục dùng DHT11 như hiện tại, nên mô tả cảm biến này là "môi trường xung quanh" thay vì "nhiệt độ ổ cắm" trong mọi tài liệu quảng bá hoặc hướng dẫn sử dụng. Việc diễn đạt đúng sẽ giúp người dùng hiểu rõ giới hạn của sản phẩm và tránh đặt kỳ vọng sai về khả năng bảo vệ an toàn điện.

Phụ lục C. Gợi ý cách sử dụng bộ sơ đồ đi kèm

Bên cạnh file báo cáo DOCX, bộ sơ đồ còn được xuất riêng dưới dạng PNG và nguồn DOT của Graphviz. Điều này giúp người dùng có thể tái sử dụng sơ đồ cho slide thuyết trình, chỉnh sửa khi bổ sung tính năng mới, hoặc nhập vào các công cụ vẽ sơ đồ khác. Khi thay đổi luồng kỹ thuật, nên cập nhật tối thiểu ba sơ đồ: use case, sequence điều khiển relay và cây dữ liệu Firebase.

Nếu mở rộng sang nhiều thiết bị hoặc nhiều tài khoản, nên bổ sung thêm actor Quản trị viên, use case Quản lý người dùng, class hoặc component cho tầng authentication, và deployment diagram mới phản ánh vai trò của backend trung gian nếu có. Bộ sơ đồ hiện tại đã được thiết kế đủ chi tiết để hỗ trợ bước chuyển đổi đó.

Phụ lục D. Danh mục ảnh giao diện và phần cứng cần bổ sung

Bảng dưới đây tổng hợp các hình ảnh nên bổ sung vào báo cáo khi hoàn thiện nghiệm thu thực tế. Danh mục này giúp bảo đảm phần minh chứng sản phẩm được trình bày đầy đủ và thống nhất với các mục đã mô tả trong Chương 4.

Bảng 26. Danh mục ảnh giao diện và phần cứng cần bổ sung.

Mã ảnh	Vị trí chèn	Nội dung nên thể hiện	Ghi chú khi chụp
Ảnh 01	Mục 4.2.1	Màn hình nhập DEVICE_ID và trạng thái kiểm tra thiết bị	Chụp khi nhập device01 hợp lệ
Ảnh 02	Mục 4.2.2	Dashboard tổng quan với trạng thái online/offline và chỉ số môi trường	Ưu tiên hiển thị đủ 4 relay
Ảnh 03	Mục 4.2.3	Màn hình điều khiển relay trước và sau khi gửi lệnh	Nên chụp trạng thái có ít nhất 1 relay đang bật
Ảnh 04	Mục 4.2.4	Danh sách lịch sử với bộ lọc đang hoạt động	Thể hiện source và timestamp của sự kiện
Ảnh 05	Mục 4.2.5	Màn hình cài đặt với các trường tên thiết bị, mode, buzzer, ngưỡng cảnh báo	Có thể chụp trước và sau khi lưu
Ảnh 06	Mục 4.3.1	Mô hình phần cứng tổng thể sau khi lắp ráp	Ảnh đủ sáng, thấy rõ relay, ESP32 và LCD
Ảnh 07	Mục 4.3.2	LCD hiển thị trạng thái vận hành hoặc cảnh báo	Có thể chụp cận cảnh để dễ quan sát